University of Frontier Technology, Bangladesh

Department of Internet of Things & Robotics Engineering

Rice Disease Detection Using DL& Soil Health Monitoring

By

Chayon Kumar Das

ID: 2001015

Suvro Kumar Das

ID: 2001021

Under the supervision of

Md Toukir Ahmed

Assistant Professor, Dept. of IRE

Bachelor of Science in Internet of Things & Robotics Engineering

University of Frontier Technology, Bangladesh

September, 2025

CERTIFICATE

This is to certify that the capstone project entitled "Rice Disease Detection Using DL& Soil Health Monitoring" by "Chayon Kumar Das", "Suvro Kumar Das" has been accepted as satisfactory in partial fulfilment of the requirements for the degree of Bachelor of Science in Internet of Things & Robotics Engineeringon September, 2025.

Signature of Supervisor

Md Toukir Ahmed
Assistant Professor
Department of Internet of Things & Robotics Engineering
University of Frontier Technology, Bangladesh

DECLARATION

We hereby declare that our capstone entitled "Rice Disease Detection Using DL& Soil Health Monitoring" is the result of our work. We also ensure that it was not previously submitted or published elsewhere for the award of any degree or diploma.

Author/Authors:

Chayon Kumar Das
ID: 2001015

Session: 2020-2021

Dept. of IRE

Suvro Kumar Das
ID: 2001021

Session: 2020-2021

Dept. of IRE

BOARD APPROVAL

This is to certify that the Capstone Project titled

"Rice Disease Detection Using DL& Soil Health Monitoring"

submitted by the group members, has been examined and approved by the Examination Board.

Examination Board:

Suman Saha Assistant Professor & Chairman Department of Internet of Things & Robotics Engineering	Md. Toukir Ahmed Assistant Professor Department of Internet of Things & Robotics Engineering
Sadia Enam Lecturer Department of Internet of Things & Robotics Engineering	Saurav Chandra Das Lecturer Department of Internet of Things & Robotics Engineering

Acknowledgement

First of all, all praises and thanks to the Almighty for bestowing His blessing upon us to complete this task.

Then we would like to express our heartfelt gratitude and respect to our honorable supervisor Md Toukir Ahmed, Assistant Professor Department of Internet of Things & Robotics Engineering, University of Frontier Technology, Bangladesh , for guidance and continuous support throughout the work.

We are grateful to other teachers and the staff of Department of Internet of Things & Robotics Engineeringfor their kind suggestions and encouragement.

Finally, we would like to thank our parents, family, friends and well-wishers for their continuous inspiration.

Chayon Kumar Das ID No: 2001015

Suvro Kumar Das ID No: 2001021

Abstract

The project will provide an automated solution to detecting rice diseases and monitoring the soil health involving cutting-edge machine learning (ML), deep learning (DL) and Internet of Things (IoT) technologies. One of the most essential crops in the world, rice is highly susceptible to diseases that severely reduce crop output and endanger food shortages, so manual methods of detecting diseases often lack accuracy and will not be viable should the agricultural scale grow to larger magnitudes. These drawbacks are being tried to overcome by this system, so the system targets a high-efficiency scalable and user friendly platform with the feature of automated disease diagnosis and soil health prediction. It is a strong and accessible solution based on Flask backend and a Jinja frontend. The system detects diseases (Rice Blast, Sheath Blight, and Bacterial Blight, etc.) using sophisticated image analytics. At the same time, the ESP32 tagged IoT system measures essential soil components such as Nitrogen (N), Phosphorus (P), Potassium (K), moisture and pH to forecast soil aptitude to yield rice and recognize the deficiency of a nutrient. The designed system has presented 97.5% accuracy in classification of a disease and correct prediction of soil health. This system will facilitate the implementation of precision agriculture, focusing on minimising disease-caused losses, maximising nutrient functionality and ensuring economic sustainability among farmers due to real-time feedback and predictive analytics through it.

Keywords: Deep Learning (DL); Internet of Things (IoT); CNN; Ensemble; Precision agriculture; Multiclass Rice Disease; Flask+Jinja; Soil Health Monitoring.

Contents

List of Figures

List of Tables

Abbreviations and List of Symbols

AI	Artificial Intelligence
API	Application Programming Interface
CNN	Convolutional Neural Network
CSS	Cascading Style Sheets
DL	Deep Learning
ESP32	A specific microcontroller
HTML	Hyper Text Markup Language
IoT	Internet of Things
ILSRVC	ImageNet Large Scale Visual Recognition Challenge
LoRa	Long Range
ML	Machine Learning
N, P, K	Nitrogen, Phosphorus, Potassium
pH	Power of Hydrogen
ReLU	Rectified Linear Unit
ROC	Receiver Operating Characteristic
AUC	Area Under Curve
RTC	Real-Time Clock
SGD	Stochastic Gradient Descent
SQL	Structured Query Language
VGG	Visual Geometry Group

Chapter 1

Introduction

1.1 Introduction

The project at hand, called *Rice Disease Detection Using DL & Soil Health Monitoring*, endeavors to solve the problem of the susceptibility of rice—the essential food staple of more than half of the global population—to diseases that considerably reduce yield, thereby endangering food security [1, 2]. The proposed project utilizes the latest image processing, machine learning (ML), and deep learning (DL) approaches to automate diagnosing diseases, increase precision, and make the diagnosis process real-time [3, 4]. The novelty of the system is to analyze soil health, which can predict whether the soil can grow rice and detect a lack of nutrients. The platform has a Flask back-end and Jinja front-end to achieve efficiency, scalability, and user friendliness.

1.2 Motivation

Plant pathogens are major threats to worldwide food security, with reports indicating that approximately 14.1% of crop production losses globally can be attributed to plant diseases, with rice being especially prone to huge losses [2, 1]. The widely occurring diseases of rice, which include Brown Spot, Blast Disease, Bacterial Leaf Blight, and Leaf Smut, are capable of decreasing yield levels by up to 20% when uncontrolled [4].

Disease detection has conventionally been performed manually by well-trained professionals, but this has been shown to be inaccurate, time-consuming, labor intensive, and impractical in large agricultural industries [3]. Moreover, agricultural experts are lacking in many regions, which makes timely disease detection and control more difficult. These shortcomings highlight the importance of developing automated and accurate solutions.

With the introduction of deep learning in artificial intelligence, a plausible way to counter these challenges has emerged, especially since it has achieved many successes in image classification tasks for detecting visual symptoms of plant diseases [5, 6]. This project is a direct response to these challenges, using advanced technologies to offer a more effective and efficient solution for the management of rice diseases.

1.3 Problem Statement

Disease detection procedures that are manual often fail to be accurate, and they cannot be used especially in extensive agricultural activities [3]. To provide on-demand diagnosis of diseases and evaluate the soil health, farmers will need an effective and accurate system that will help them maintain food security [2]. When there is no presence of such an automated system, there is substantial reduction of yield caused by unchecked diseases and unoptimised nutrient conditions [1].

1.4 Objectives

The main goals of the given project revolve around increasing the productivity and sustainability of agriculture. The first is to make disease diagnosis automated, by utilizing advanced techniques of image processing, machine learning (ML), and deep learning (DL) to characterize rice diseases such as Rice Blast, Sheath Blight, and Bacterial Blight [4, 5]. Second, it revolves around the prediction of soil health using an IoT system based on ESP32 which tracks the main elements of the soil, such as Nitrogen (N), Phosphorus (P), Potassium (K), soil moisture, and pH, therefore, establishing whether soil is suitable to grow the rice crop or not, and which components are lacking in the soil [7]. Moreover, the project aims at establishing a scalable and user-friendly system integrating Flask as a backend and Jinja as a frontend to make it flexible with respect to different crops and geographical zones. Finally, it also seeks to facilitate precision agriculture by giving a real-time intuitive feedback to farmers and predictive analytics to minimize losses due to diseases and perform better nutrition management [8].

- **Automate Disease Diagnosis:** Detection of rice diseases including Rice Blast, Sheath Blight and Bacterial Blight can be identified using Machine Learning (ML) and Deep Learning (DL) [4, 5, 6].
- **Predict Soil Health:** Install an IoT system composed of an ESP32 to evaluate Nitrogen (N), Phosphorus (P), Potassium (K), moisture and pH as well as to identify the suitability and nutrient deficiencies of the soil [7].
- **Scalable Platform:** Build a backend that is based on Flask and a frontend based on Jinja to implement a system which will handle various crops and geographical areas.
- **Precision Agriculture:** Offer predictive analytics and real-time feedback to enable farmers to minimize crop losses and optimize nutrient management [8].

Chapter 2

Background Study

Chapter 3 has given an overview of the major tools applied in agricultural monitoring systems in terms of rice disease monitoring and soil health, and then discussed the models of deep learning in automatic analysis.

2.1 Hardware & Sensors

2.1.1 ESP32 Microcontroller

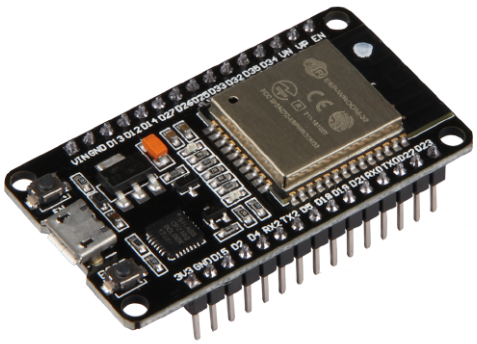

Figure 2.1: ESP 32 Dev Module

The central control panel of the system is the ESP32 DevKit V1. It has a 32-bit Tensilica dual-core processor (up to 240 MHz) and onboard 802.11 b/g/n Wi-Fi and Bluetooth Low Energy (BLE) so that sensor data can be wirelessly sent to local or cloud-based services. The board has several interfaces, GPIO, UART, I^2C, SPI, and 12-bit ADC channels, which means that it can accept analog sensor voltages and digital serial data. The usual I/O voltages are 3.0–3.3 V (powered over 5 V USB or external regulated supply). In this project the ESP32 will be used to carry out sensor data capture, basic sensor data preprocessing (filtering and calibration), RS485-to-TTL digital sensor interfacing, and communication with the backend [7].

2.1.2 Capacitive Soil Moisture Sensor

Figure 2.2: Soil Moisture Sensor

Unlike resistive probes, the capacitive soil moisture sensor detects changes in dielectric permittivity to measure volumetric water content; this has the benefit of minimal corrosion over the long term, making it suitable for field use. It normally operates in the range of 3.3–5 V and provides an analog output that reflects humidity levels, which is read through an ESP32 ADC input. Upon calibration, the ADC value may be scaled to a 0–100% moisture scale. Since the ESP32 ADC is non-linear at its extremes, a simple calibration curve (two-point or polynomial fit) is recommended to translate raw ADC values into percentage moisture.

2.1.3 Soil pH Sensor (DFRobot Gravity: Analog pH Sensor Kit V2)

Figure 2.3: Gravity analog pH sensor

The DFRobot Gravity: Analog pH Sensor Kit V2 is a soil and water pH measurement module designed for microcontroller integration. It uses a BNC-type probe connected to a signal

conditioning board that amplifies and filters the electrode's millivolt-level signal, providing a stable analog voltage (0–3.0 V) suitable for ESP32 ADC input. The kit operates at 3.3–5 V and supports a measurement range of pH 0–14, with accuracy typically ±0.1 pH (at 25 °C). Calibration is performed using standard buffer solutions (commonly pH 4.0 and pH 7.0), ensuring reliable long-term measurements. Regular cleaning and recalibration of the probe are necessary to maintain accuracy, especially in outdoor or agricultural soil environments.

2.1.4 NPK Sensor (RS485 / UART)

Figure 2.4: NPK sensor

NPK sensors measure soil nutrient concentrations (Nitrogen, Phosphorus, Potassium) using ion-selective or electrochemical methods and output processed results digitally. The modules typically use the RS485 (Modbus RTU) protocol and operate on 5–12 V. An RS485-to-TTL converter (e.g., MAX485) is used to interface with the ESP32; the converter connects to ESP32 UART pins for serial communication. Concentrations are usually reported in mg/kg (ppm), with sensor firmware providing JSON or register-based payloads including N, P, K values and status codes. Because of digital communication, NPK sensors are robust to cable noise and can be networked in field deployments [7].

2.1.5 DHT22 Temperature and Humidity Sensor

Figure 2.5: DHT 22:Temperature & humidity sensor

The DHT22 (AM2302) is a low-cost digital sensor that measures ambient temperature and relative humidity. It operates on a 3.3–5 V supply and communicates using a proprietary single-wire digital protocol. The sensor provides 16-bit temperature data (accuracy ±0.5 °C) and 16-bit humidity data (accuracy ±2–5%). Typical measurement range is –40 to 80 °C for temperature and 0–100 %RH for humidity. Data is read by the ESP32 GPIO pin using a timing-based library, which decodes the pulse-width modulated signal. A pull-up resistor (4.7–10 kΩ) is required between the data pin and VCC for reliable communication. Due to its digital output, the DHT22 is resilient against moderate signal noise, making it suitable for indoor and outdoor agricultural monitoring applications.

2.1.6 Power System and Enclosure

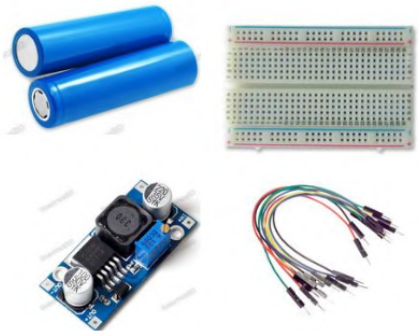

Figure 2.6: 18365 Battery, Jumper Wires,DC-DC Boost Module & Breadboard

The system can be powered in two ways: (i) using a 5 V USB adapter for laboratory testing and debugging, or (ii) using three 18650 Li-ion cells (3.7 V nominal each) for portable field operation. A DC–DC booster module regulates the battery voltage to a stable 5 V supply

required by the electronics. For prototyping, two standard breadboards with jumper wires are used for circuit integration. A toggle switch provides convenient on/off control, while all components are assembled inside a weather-resistant enclosure to ensure durability against dust, rain, and sunlight.

2.2 Models

2.2.1 Convolutional Neural Networks (CNNs)

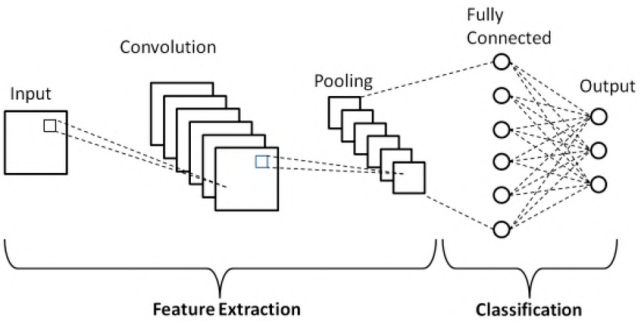

Figure 2.7: CNN Architecture

Detection of images of diseases in CNNs is extensively based on image-based CNN feature maps that feature hierarchical structures via convolution and pooling layers. Their learning abilities on complicated patterns makes them so powerful in being used in diagnosis of the rice diseases symptoms [3, 4].

2.2.2 VGG

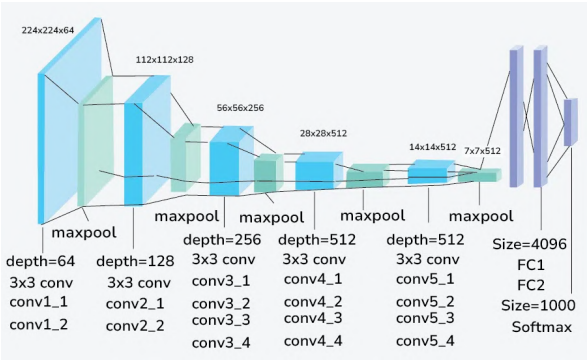

Figure 2.8: VGG Architecture

Those models use deep n-3x3 conv features, they have strong capability to catching feature. Being computationally expensive, the VGG architecture proves useful in the area of classification, such as disease detection of the Basal Stem Rot with approximately 91.93% accuracy [6, 9].

2.2.3 ResNet (Residual Networks)

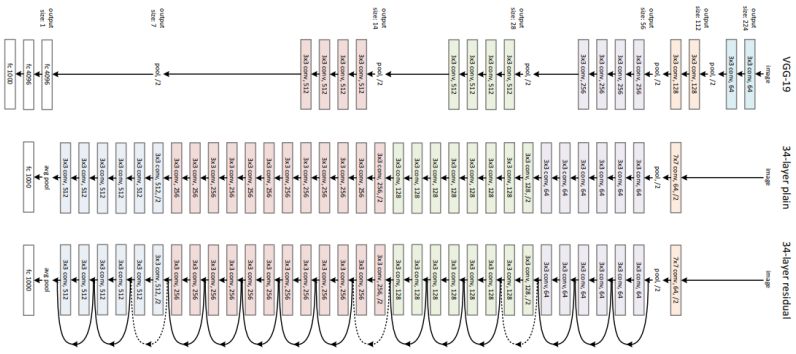

Figure 2.9: ResNet Architecture

ResNet has residual (skip) connections that assist in training very deep networks, by alleviating the vanishing gradient problem. ResNet-101 show quite decent accuracy in rice leaves disease classification [5, 10].

2.2.4 GoogLeNet / Inception Models

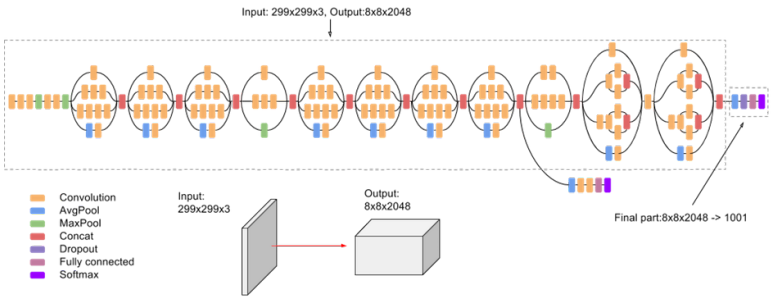

Figure 2.10: Inception Architecture

Inception architecture executes the parallel convolutions on the same kernel size of several sizes within the blocks thus learning multi-scale features concurrently. Inception-ResNet-V2 augments this with residual connections and was able to achieve high level of accuracy in detecting rice disease [5, 6].

2.2.5 MobileNet Architectures

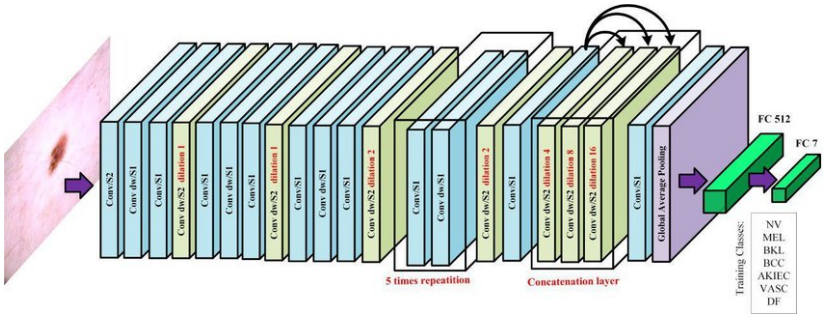

Figure 2.11: MobileNet Architecture

Developed in mobile and edge computing, MobileNet applies depthwise separable convolutions to radically decrease the mass and computations, is realistic to execute in a restricted device at real-time and competitive accuracy [11, 12].

2.2.6 EfficientNet B4

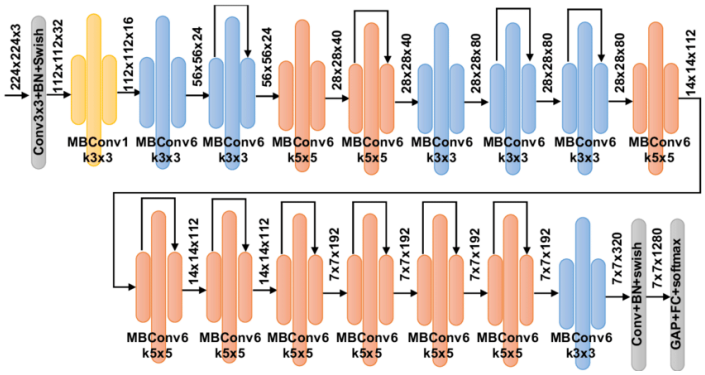

Figure 2.12: EfficientNet B4 Architecture

EfficientNet can use compound scaling of depth, width, and resolution and optimise model size and accuracy. B4 version is a excellent trade in the number of resources and excellent levels of classification [5, 13].

2.2.7 Xception Networks

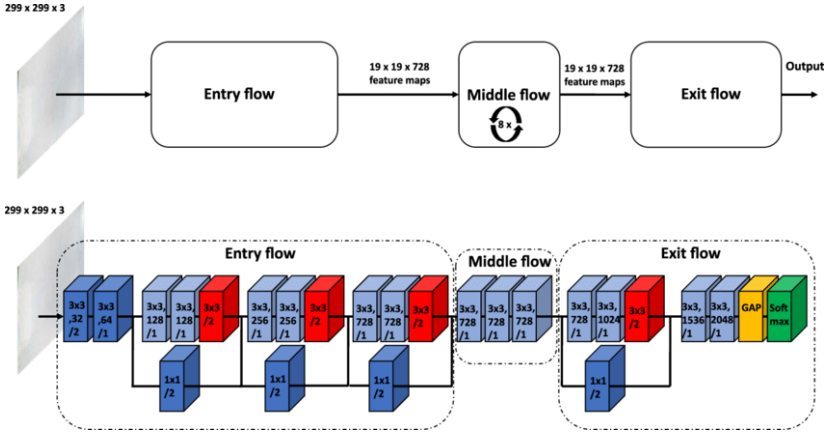

Figure 2.13: Xception Architecture

Xception is an improvement over traditional convolutions using Depth wise separable convolutions for a faster computation. Transfer learning is often done with a lot of rice disease classification tasks [5, 9].

2.2.8 DenseNet Models

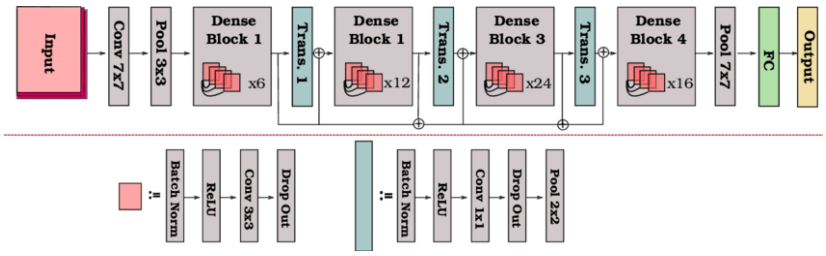

Figure 2.14: DenseNet Architecture

DenseNet links every layer to all others within a dense block so that feature reuse and gradient flow are enhanced. The architecture minimizes the parameters and maximizes the performance of the disease detection in plants [6, 14].

2.2.9 SqueezeNet

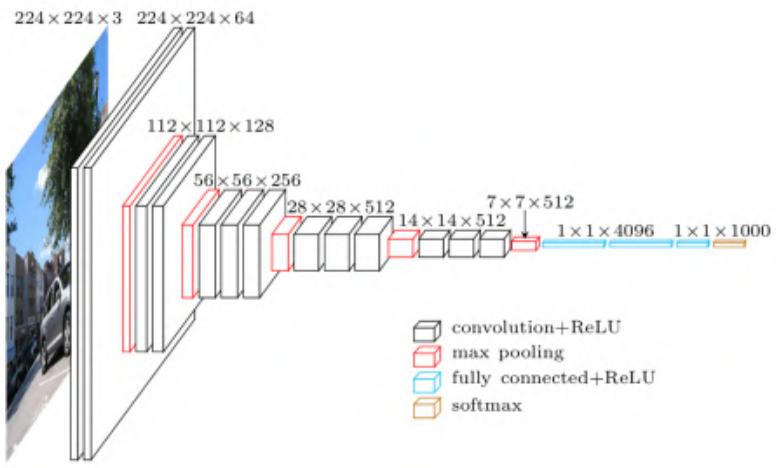

Figure 2.15: SqueezNet Architecture

SqueezeNet also has a similar performance to AlexNet with a smaller number of parameters through the use of "Fire Modules" to squeeze and expand feature maps giving it the ability to be deployed as a lightweight object [12, 11].

2.3 Performance Metrics and Evaluation

To assess the quality of a deep learning model in classifying rice diseases one needs to know a number of performance metrics that are based on the confusion matrix. Confusion matrix is used to summarize prediction results based on actual and predicted classes through which measures like accuracy, precision, recall, and F1- score can be calculated [3, 5].

2.3.1 Confusion Matrix

$$\begin{bmatrix} \text{True Positive (TP)} & \text{False Positive (FP)} \\ \text{False Negative (FN)} & \text{True Negative (TN)} \end{bmatrix}$$

- *True Positive (TP)*: Number of correctly classified positive samples (e.g., correctly detected diseased leaves). - *False Positive (FP)*: Number of incorrectly classified positive samples (healthy samples wrongly detected as diseased). - *False Negative (FN)*: Number of positive samples incorrectly classified as negative. - *True Negative (TN)*: Number of correctly classified negative samples.

2.3.2 Accuracy

Accuracy measures the overall correctness of the model:

$$\text{Accuracy} = \frac{TP\ TN}{TP\ TN\ FP\ FN}$$

It indicates the proportion of total correct predictions but may be misleading in imbalanced datasets [2].

2.3.3 Precision

Precision quantifies the correctness of positive predictions:

$$\text{Precision} = \frac{TP}{TP\ \ FP}$$

High precision means few false positives, important when false alarms are costly [1].

2.3.4 Recall (Sensitivity)

Recall measures the ability to detect all actual positives:

$$\text{Recall} = \frac{TP}{TP\ \ FN}$$

High recall indicates few false negatives, crucial in disease detection to avoid missing cases [4].

2.3.5 F1-Score

The F1-score balances precision and recall as their harmonic mean:

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision}\ \ \text{Recall}}$$

It provides a single metric to evaluate performance when classes are imbalanced [6].

2.3.6 Mean and Standard Deviation

When assessing models across multiple runs or cross-validation folds, statistical measures such as the mean and standard deviation of performance metrics are reported to summarize variability [15].

Given a set of values $x_1, x_2, ..., x_n$ (e.g., accuracies from n runs):

$$\text{Mean } \mu = \frac{1}{n} \sum_{i=1}^{n} x_i$$

$$\text{Standard Deviation } \sigma = \sqrt{\frac{1}{n-1} \sum_{i=1}^{n} x_i - \mu^2}$$

Reporting $\mu \pm \sigma$ conveys the average performance and its consistency.

2.3.7 Example Application

Suppose a rice disease classification model outputs the following confusion matrix for a test set of 100 samples:

$$\begin{bmatrix} 45 & 5 \\ 10 & 40 \end{bmatrix}$$

Calculations:

$$\text{Accuracy} = \frac{45\ \ 40}{100} = 0.85 = 85\%$$

$$\text{Precision} = \frac{45}{45\ \ 5} = 0.90 = 90\%$$

$$\text{Recall} = \frac{45}{45\ \ 10} = 0.818 = 81.8\%$$

$$F_1 = 2 \times \frac{0.90 \times 0.818}{0.90\ \ 0.818} \approx 0.857 = 85.7\%$$

This model shows good precision and balanced recall, resulting in a strong F1-score.

These metrics guide model selection and tuning in automated rice disease detection systems to optimize accuracy while minimizing misclassification costs [16].

Chapter 3

Literature Study

3.1 Literature Review

Next, several researchers have studied the detection of rice leaf diseases using different deep learning and machine learning approaches. Aman et al. [6] tested and compared four models; LeafNet, a Modified LeafNet, MobileNetV2, and the Xception, on 2658 images depicting healthy and diseased rice leaves. The best performance occurred in the Modified LeafNet, representing a modified architectural parameters that obtained a validation accuracy of 97.44% and the testing accuracy of 87.76% establishing the future potential of the customized architecture in the agricultural domain of application.

There exists a literature review of problematic issues related to meeting the challenges during the combination of deep learning and edge computing in rice leaf diseases detection, including the necessity of a real-time model deployment as well as the detection of small and overlapping lesions [8]. The researchers concluded that the YOLOv7 model was the most accurate (99 percent mean Average Precision (mAP)) and tweeted that edge computers such as NVIDIA Jetson Nano were ideal devices to use in the field of agriculture.

Improved deep neural networks have also been suggested in promoting paddy leaf disease classification when implementing precision agriculture. Another study [17] used some optimized DNN model and reached 95% accuracy, but not all architectural details thereof were provided. In yet another study [18], the CNN-based transfer learning models VGG-19, Inception-ResNet-V2, ResNet-101 and Xception, achieved accuracy of 92.68% in a total of 4 types of diseases and healthy leaf category.

Survey-based studies have provided comprehensive overviews of the field. Orchi et al. [19] reviewed methods using machine learning, deep learning, image processing, IoT, and hyperspectral imaging for crop disease detection, categorizing them into direct and indirect approaches and identifying gaps for future research. Another article [20] focused specifically on image segmentation techniques for identifying diseased plant regions, emphasizing their importance in preprocessing for classification tasks.

New architectures have been presented in the detection of rice leaf disease. PlantDet model, a multi-model ensemble with InceptionResNetV2, EfficientNetV2L and Xception had

very good performance of 98.53% on rice leaf classification and with good generalizability by being used in analysis of the betel leaf diseases [5]. Other examples of plant species; other than rice, a study on oil palm seedlings [21] has conducted hyperspectral imaging and Mask RCNN to segment Basal Stem Rot (BSR) symptoms with the VGG16 model attaining an accuracy of 91.93%.

Additional studies have been revolving around the formulation of CNN based models that can perform in a complex set of environments. An example of models [22] was to categorize diseases in rice leaves in images with different light and various backgrounds, achieving 95 percent accuracy. There has also been an exploration of targeted disease detection formerly Faster R-CNN on rice false smut identification [23] which combines regional proposal contrast and object detection in one network.

It has been pointed out in some review studies such as [2], that deep learning models have become more effective in plant disease detection, and their potential usefulness mentioned in the review includes CNNs, transfer learning, multimodal fusion, data augmentation and hardware optimization as most approaches aim to achieve an accuracy of more than 96 percent. The methodology of classifying the rice blast disease using Artificial Neural Network (ANN) that scored 90% and 86% in testing infected samples and healthy ones respectively in the previous years was previously the best examples especially in its application of machine learning that have preceded their increasingly advanced deep learning approaches in recent years [24].

3.2 Comparative Study

The section has done a comparative analysis of machine learning and deep learning methods of rice disease classification as it has been noted in recent literature. Deep learning methods always perform better than the traditional machine learning algorithms, in terms of accuracy, robustness and scalability. As in the illustration of an optimized Deep Neural Network (DNN) with a Crow Search Algorithm wise 95 percent accuracy which is very much better when compared to Support Vector Machine (SVM) index which has very slight percent value to less than 75 [17]. Transfer learning methods applied with architectures Inception-ResNet-V2, ResNet-101, VGG-19, Xception and EfficientNet-variants typically perform better, with ensemble prediction methods such as PlantDet having accuracy up to 98.53% [5].

The insights from the review articles is that accuracies over 96% can be obtained using a combination of effective preprocessing, data augmentation (e.g., Dual GAN-based augmentation leading to a near 4.5-percentage accuracy increase), and more advanced neural network structures [2]. In addition, practically performing on-field detection of disease in real-time, and maintaining optimality of performance has been achieved, through integration of edge computing hardware [8].

To conclude these papers we found that the various deep learning models and ensemble

Table 3.1: Literature Review of Rice Leaf Disease Detection Approaches

Paper Title	Authors	Year	Method / Model	Accuracy	Key Contributions
Rice Plant Disease Detection using Convolutional Neural Networks [25]	A.B. Ayyappan et al.	2025	CNN Models	97.5%	Transfer Learning with multiple CNN weights for preprocessing
Deep Learning and Edge Computing in Agriculture: A Comprehensive Review of Recent Trends and Innovations [8]	Junaidi et al.	2025	YOLOv7, ViT, Edge AI	99% mAP	Review of DL + edge computing for rice disease detection
Comparative Analysis of Transfer Learning, LeafNet, and Modified LeafNet Models for Accurate Rice Leaf Diseases Classification [6]	Altabaji et al.	2024	LeafNet, Modified LeafNet, MobileNetV2, Xception	97.44%	Modified LeafNet outperformed CNN and TL models on rice leaves
Resource-optimized CNNs for Real-time Rice Disease Detection with ARM Cortex-M [26]	Nugroho et al.	2024	MobileNetV2, FD-MobileNet	97.5% / 90%	Microcontroller-friendly rice disease detection pipeline
Hybrid DC-GAN-MDFC-ResNet in IoT framework [27]	Wang et al.	2024	DC-GAN + ResNet	88.66–91.86%	Disease localization + classification under varied conditions
PlantDet: A Robust Multi-Model Ensemble Method Based on Deep Learning for Plant Disease Detection [5]	Shovon et al.	2023	InceptionResNetV2, EfficientNetV2L, Xception (Ensemble)	98.53%	Robust ensemble model with Grad-CAM explainability
Automatic Disease Detection of Basal Stem Rot Using Deep Learning and Hyperspectral Imaging [21]	Yong et al.	2023	VGG16, Mask R-CNN, Hyperspectral	91.93%	Hyperspectral + DL for BSR detection in oil palm
Multispectral + RGB DL pipeline for rice disease detection [28]	Alnaggar et al.	2023	Deep CNN on multispectral + RGB	Higher F1 than RGB baseline	Public dataset with multispectral images
On Using Artificial Intelligence and the Internet of Things for Crop Disease Detection: A Contemporary Survey [29]	Orchi et al.	2022	AI + IoT, ML + DL survey	N/A	Contemporary survey of AI and IoT in crop disease detection
Rice Leaf Disease Classification and Detection Using YOLOv5 [30]	Haque et al.	2022	YOLOv5	Precision 90%, Recall 67%, mAP 76%	First annotated dataset for rice leaf object detection
Automatic Detection of Rice Disease in Images of Various Leaf Sizes [31]	Kiratiratanapruk et al.	2022	CNN + leaf-size–aware tiling (ResNet18)	mAP 91.1%	Handles variable leaf sizes in real-field images
Paddy Leaf Disease Detection Using an Optimized Deep Neural Network [17]	Nalini et al.	2021	DNN + Crow Search Optimization	96.96%	CSA optimization improved rice disease classification
An Automated Convolutional Neural Network Based Approach for Paddy Leaf Disease Detection [32]	Islam et al.	2021	VGG19, ResNet101, InceptionResNetV2, Xception	92.68%	Automated CNN-based rice disease classification
Rice False Smut Detection Based on Faster R-CNN [23]	Sethy et al.	2020	Faster R-CNN (ResNet50 backbone)	N/A	First attempt at RFS detection using Faster R-CNN
Rice Blast Disease Detection and Classification Using Machine Learning Algorithm [24]	Ramesh & Vydeki	2018	ANN + Feature Extraction (GLCM)	Train: 99%, Test: 90%	Early blast detection using ANN
Crop Disease Detection Using Image Segmentation [20]	Jaware et al.	2012	K-Means Clustering, Image Segmentation	N/A	Classical segmentation approach for crop disease detection

models were applied in the best manner. Data augmentation and preprocessing still plays important role of enhancing and generalizing models among various other models especially when there are small dataset. With the development and enhancement of edge computing which in turn is capable of deployment of such models in real-time agricultural environments supporting precision agriculture applications [8].

3.3 Research Challenges

Although the deep learning approach has made immense progress in solution of plant disease detection, still major problems stand out that impede successful implementation and popularization of the application:

1. **Dataset Limitations:** A significant lack of big, varied, and correctly labeled databases is observed that reflect the total variability in diseases in natural settings. Current datasets are small and depend on quality images thus limiting the model generalizability and the real world applicability [15, 16].

2. **Generalization Issues:** There are also cases where the models fail to perform at high levels when they are implemented into field conditions. The use of variable light, complex backgrounds, occlusions and similar looking diseases all contributes to the lower accuracy in non controlled environments [22, 23].

3. **Computational Constraints:** Deep learning models are computation-intensive with scalability that is cumbersome to execute real-time inference on edge devices in the agricultural field, which have limited resources. This adds to lack of accessibility and scalability since there is a need to use specific hardware [8, 11, 12].

3.4 Research Gaps

To advance the field of rice disease detection, the following gaps must be addressed:

- **Dataset Development:** Production of vast, heterogeneous, and fully annotated datasets that faithfully reflect the diversity in the real world such as diverse stages of diseases, environmental factors and varieties of rice [15, 16].

- **Class Imbalance Mitigation:** Addressing imbalance of the dataset to minimize the bias in the model such as balancing the number of examples of each particular class by increasing the number of the under-populated classes to obtain equal numbers of examples of each class, e.g. standardizing over all of the classes to 1000 images [2, 13].

- **Robustness and Generalization:** Increasing the model robustness to unconstrained deployment such as changing illumination, occlusions, and cluttered backgrounds so that it can perform consistently [22, 23].

- **Fine-grained Disease Differentiation:** Improves the capacity of the models to differentiate between very similar disease in terms of their visual manifestations [9, 33].
- **Model Efficiency for Edge Deployment:** Inventing the lightweight computationally intensive models, through which it is possible to detect the disease in real time on resource-limited devices, and therefore implement practical real-life applications on the ground [8, 11, 12].

Chapter 4

Methodology and System Architecture

This chapter outlines the methodologies and the design of the system used in the project M & L Rice Disease Detection and Monitoring Soil Health. It summarizes the major existing methodologies in the plant disease detection using deep learning techniques, and subsequently, it has given the detailed mechanism behind proposed methodology under the hardware and software system architecture.

4.1 Existing Methodology

In past literature, researchers have discussed different deep learning and machine learning methods to enhance the accuracy and efficiency of rice diseases. There are two representative ways which have been brought to the fore.

4.1.1 Method 1: Paddy Leaf Disease Classification Using an Optimized Deep Neural Network

This method uses a Deep Neural Network (DNN) optimized by the Crow Search Algorithm (CSA) to improve classification [17].

- **Approach:** Initially image pretreatment and feature extraction was carried out subsequently as an input to the DNN faded into place. And secondly CSA is superior in optimization over DNN weights and biases during training to improve convergence.
- **Achievements:** Accuracy reportedly touched 96.96% and the precision and recall were above 95%. Comparatively, a classic Support Vector Machine (SVM) model obtained 73.33% of the accuracy.
- **Limitations:** Large annotated datasets are required; specific challenges such as overfitting and environmental variability remain unaddressed.

4.1.2 Method 2: Automated CNN-Based Paddy Leaf Disease Detection Using Inception-ResNet-V2

This method leverages state-of-the-art CNN architectures with transfer learning on a curated dataset [18].

- **Approach:** Transfer learning is applied to models such as VGG-19, ResNet-101, and Inception-ResNet-V2, training on 984 images.
- **Achievements:** Inception-ResNet-V2 attained the highest accuracy of 92.68%, followed by ResNet-101 (91.52%) and VGG-19 (81.43%).
- **Limitations:** Limited dataset size and dependency on high-quality images restrict deployment in real-world field conditions.

4.2 Proposed Methodology

Our proposed system integrates machine learning and deep learning models with an IoT-based soil monitoring setup to provide comprehensive rice disease diagnosis and soil health analysis. This modular architecture supports scalability and real-time operation [8, 7].

4.2.1 Automated Disease Diagnosis

State-of-the-art image processing and deep learning models are employed to detect multiple rice diseases including Rice Blast, Sheath Blight, and Bacterial Blight [4, 5]. Key features include:

- Training and evaluation of diverse CNN architectures: EfficientNet-B4, DenseNet121, Xception41, MobileNetV3 Large, InceptionV3, AlexNet, VGG19, ResNet50, ShuffleNet, SqueezeNet, and an Improved LeNet-5 [6, 13, 14].
- Use of transfer learning with pretrained weights (ImageNet).
- Data augmentation (random horizontal flip, rotation, resized crop, color jitter) to enhance robustness [2].
- Optimization using Adam optimizer, learning rate schedulers (StepLR, ReduceLROnPlateau), and automatic mixed precision (AMP) for efficient training.
- Early stopping and model checkpointing based on validation performance.
- Achieved classification accuracy exceeding 97.5% across diverse environmental conditions [22].

4.2.2 Soil Health Monitoring (IoT System)

An ESP32-based sensor suite monitors critical soil parameters in real time:

- Sensors include capacitive soil moisture, gravity analog soil pH (BNC type), RS485 NPK sensor interfaced via MAX485 converter.
- Real-time data collection, transmission, and processing enable nutrient deficiency detection and soil suitability analysis [7].

Hardware Setup

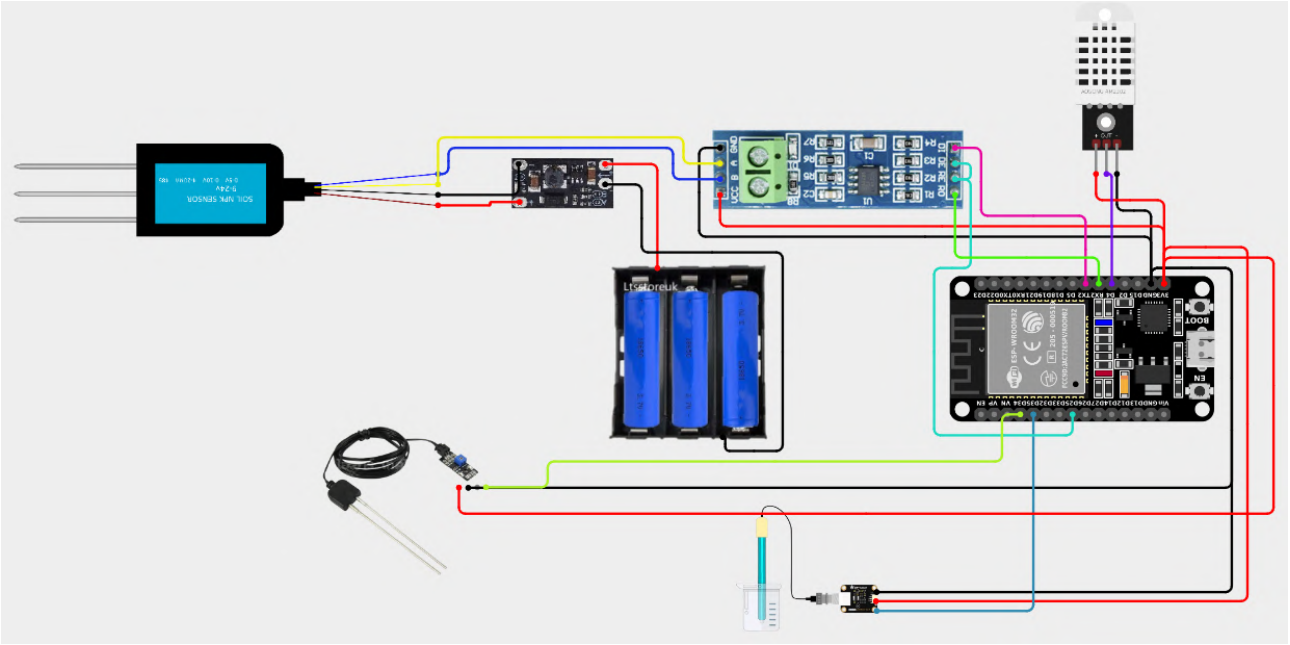

Figure 4.1: Hardware Setup

4.2.3 Data Preprocessing

This is done by using the Paddy Doctor: Paddy Disease Classification dataset on Kaggle that consists of 11,420 images of 10 classes of diseases. The analysis of the first count revealed a large class skew such that, some classes had too few images than other classes and others had more images than the average.

To address this imbalance and ensure robust model training, the following steps were taken:

- Classes with fewer than 1000 images were augmented using techniques such as random horizontal flips, rotations (up to 15 degrees), color jitter, and affine transformations to increase their sample size to 1000 images.
- Classes with more than 1000 images were randomly downsampled to exactly 1000 images to maintain uniformity across classes.

The resulting balanced dataset consists of 10,000 images, with 1000 images per class [15, 16].

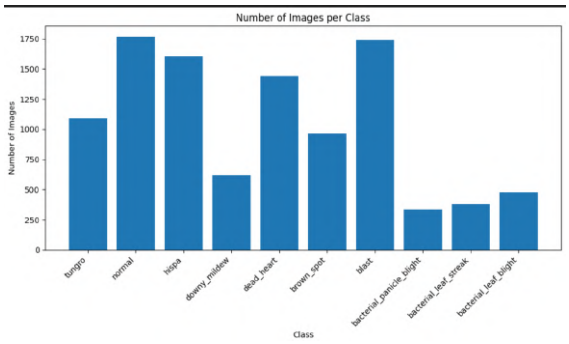

Figure 4.2: Class Distribution Before Balancing

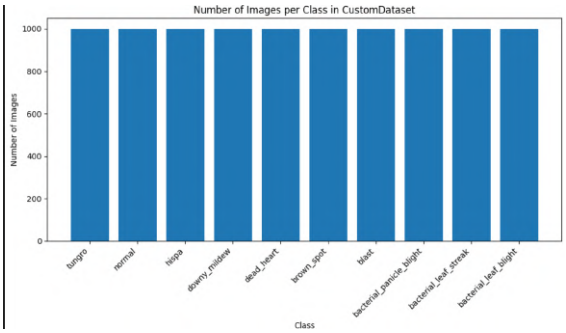

Figure 4.3: Class Distribution After Balancing

Figures 4.2 and 4.3 illustrate the class distribution of the dataset before and after balancing, respectively. The balanced dataset ensures equal representation of each class, which helps in mitigating model bias and improving classification performance.

The resulting dataset contains 10,000 images (1000 per class), ensuring robustness during model training and evaluation [2].

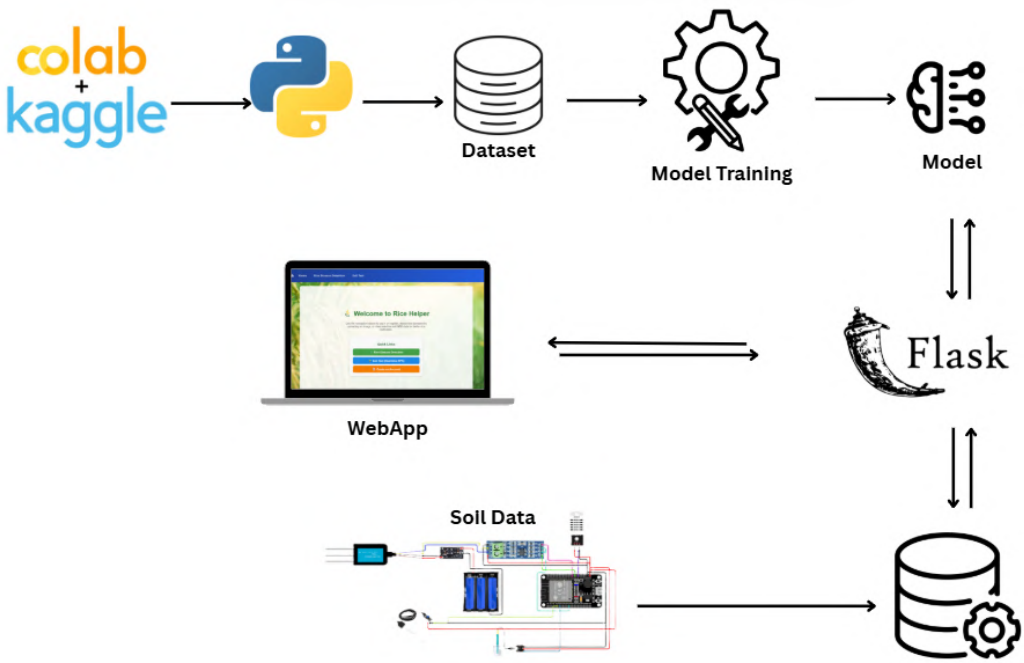

Figure 4.4: Full Project Setup

4.3 Summary

The approach is appropriate to integrate complex machine learning models designed to detect the presence of a disease in the image with the opportunity to monitor real-time soil health with the help of the Internet of Things [5, 7]. High accuracy and scalability in the form of precision agriculture are achieved related to the balanced dataset and training protocols which imply the rigorous training routines and modular architecture [15, 2, 8].

Chapter 5

Deep Learning Model Implementation and Training

5.1 Data Preprocessing for Deep Learning Models

Before training, the dataset was balanced and preprocessed to ensure fairness and robustness:

- Dataset Balancing via Augmentation for subclasses with fewer than 1000 images (e.g., `downy_mildew` with 620 images, `bacterial_panicle_blight` with 337 images).
- Limiting Overrepresented Classes by random deletion for classes with more than 1000 images (e.g., `normal` with 1767 images).

[15, 2, 16]

5.2 General Deep Learning Training Procedures

A standardized training protocol was applied across models:

- **Data Preparation and Splitting:** 70% training, 15% validation, 15% testing.
- **Data Augmentation:** Additional augmentations during training including horizontal flips, rotations, resized crops, and color jitter [13].
- **Normalization:** Using ImageNet statistics or custom values depending on the model.
- **Transfer Learning:** Utilizing pretrained weights from ImageNet [6].
- **Custom Classifier Heads:** Adjusted for 10-class classification.
- **Automatic Mixed Precision (AMP):** For efficient training.
- **Loss Function:** CrossEntropyLoss for multi-class classification.
- **Optimizer:** Adam with learning rates of 0.0001 (or 0.001 for LeNet-5).
- **Learning Rate Scheduling:** StepLR or ReduceLROnPlateau.
- **Early Stopping:** To prevent overfitting (patience 5–10 epochs).

- **Checkpointing:** Saving best model weights.
- **Testing and Saving:** Final evaluation on test set and saving models.
- **GPU Memory Management:** Calls to `gc.collect()` and `torch.cuda.empty_cache()` before training sessions.

5.3 Specific Deep Learning Model Implementations and Achievements

Several architectures were trained and evaluated with the following highlights:

- **EfficientNet-B4:**
  - Transfer learning with pretrained ImageNet weights.
  - Data augmentation and normalization.
  - Adam optimizer, StepLR scheduler, AMP, early stopping.
  - Final test accuracy: 97.13%.
- **Xception41:**
  - Transfer learning with custom normalization.
  - AMP, Adam optimizer, StepLR scheduler.
  - Final test accuracy: 97.07%.
- **DenseNet121:**
  - Pretrained DenseNet121 with adjusted classifier.
  - AMP, Adam optimizer, StepLR scheduler.
  - Final test accuracy: 97.20%.
- **DenseNet201:**
  - Similar setup to DenseNet121.
  - Final test accuracy: 97.20%.
- **MobileNetV3 Large:**
  - Pretrained model with classifier adaptation.
  - AMP, Adam optimizer, StepLR scheduler.
  - Final test accuracy: 96.73%.

- **InceptionV3:**
  - Modified final and auxiliary classifier layers.
  - Data augmentation and ImageNet normalization.
  - AMP, Adam optimizer, StepLR scheduler.
  - Final test accuracy: 96.53% .
- **AlexNet:**
  - Pretrained AlexNet with adjusted output layer.
  - AMP, Adam optimizer, StepLR scheduler.
  - Final test accuracy: 95.60%.
- **Improved LeNet-5:**
  - Three convolutional layers, batch normalization, ReLU, max pooling, dropout.
  - Data augmentation and normalization.
  - Adam optimizer with ReduceLROnPlateau scheduler.
  - Final test accuracy: 86.67%.
- **Ensemble Model :**
  - Combines four state-of-the-art CNN architectures (EfficientNetB4, DenseNet121, InceptionV3, MobileNetV3-Large).
  - Each model is fine-tuned on the rice disease dataset with transfer learning, batch normalization, and dropout to reduce overfitting.
  - Soft-voting (probability averaging) is applied to aggregate predictions from all base models.
  - Data preprocessing includes normalization, augmentation (rotation, flip, shift), and resizing to fit backbone model input dimensions.
  - Ensemble improves robustness and reduces variance compared to single models.
  - Final test accuracy: 97.50% | Loss: 0.5762.

5.3.1 Train vs Validation Accuracy Graphs

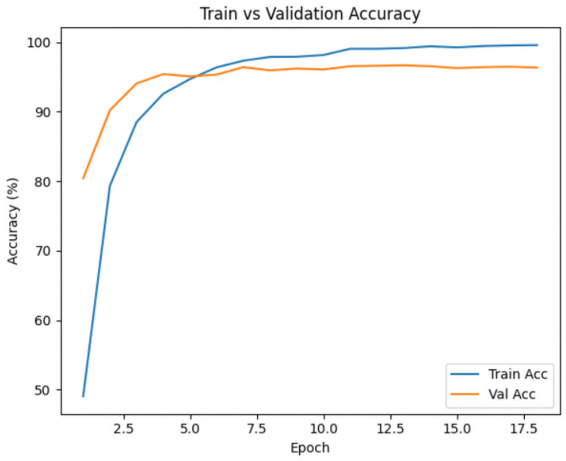

Figure 5.1: EfficientNet-B4 Train Accuracy Vs Validation Accuracy

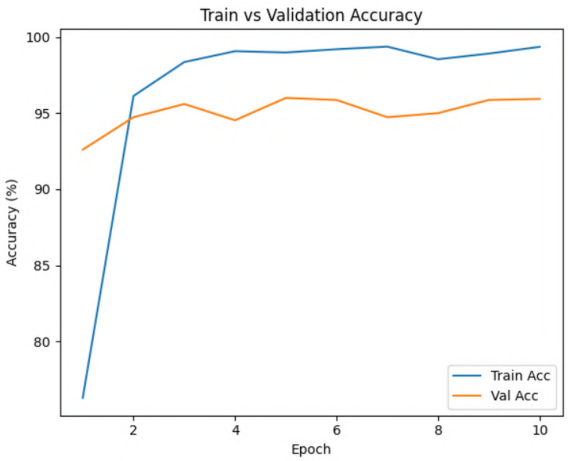

Figure 5.2: DenseNet121 Train Accuracy Vs Validation Accuracy

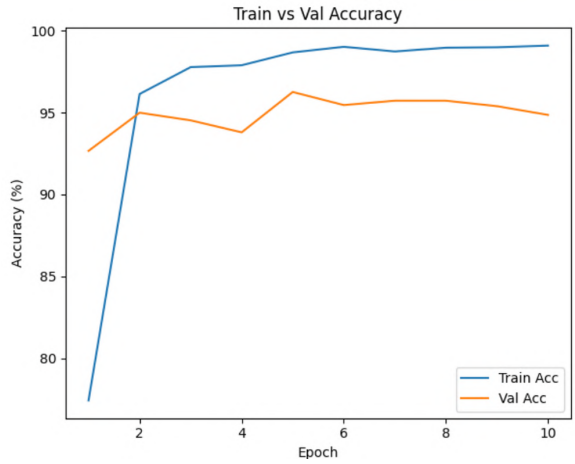

Figure 5.3: Xception41 Train Accuracy Vs Validation Accuracy

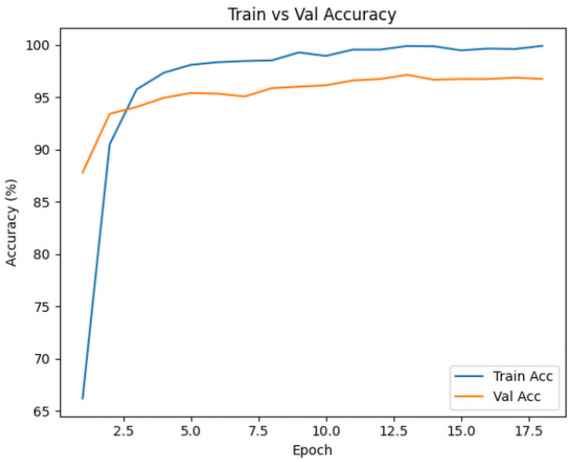

Figure 5.4: MobileNetV3 Large Train Accuracy Vs Validation Accuracy

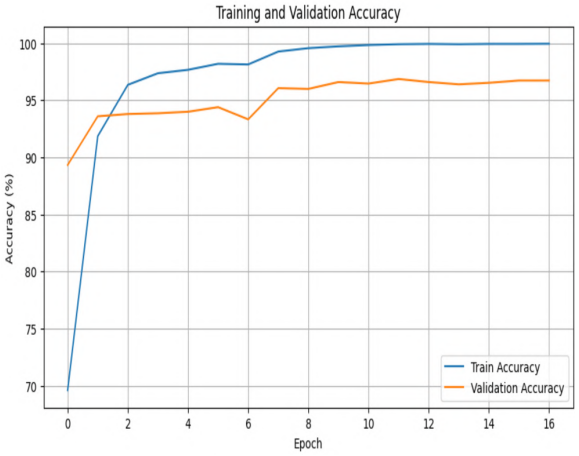

Figure 5.5: InceptionV3 Train Accuracy Vs Validation Accuracy

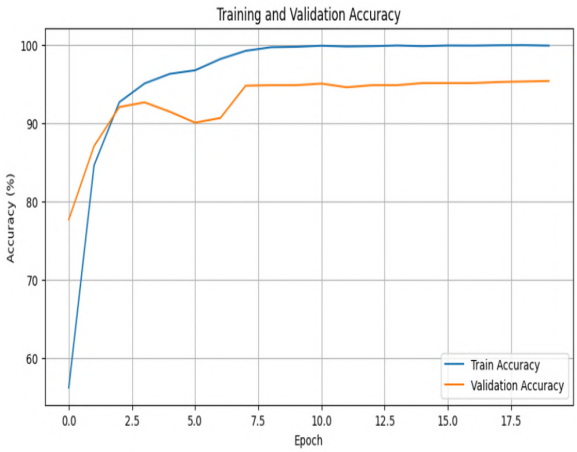

Figure 5.6: AlexNet Train Accuracy Vs Validation Accuracy

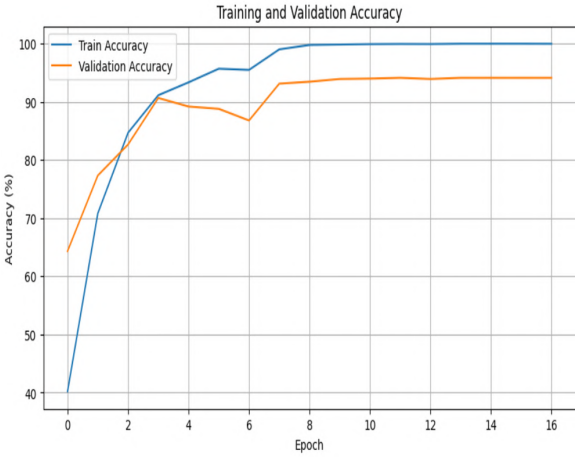

Figure 5.7: VGG19 Train Accuracy Vs Validation Accuracy

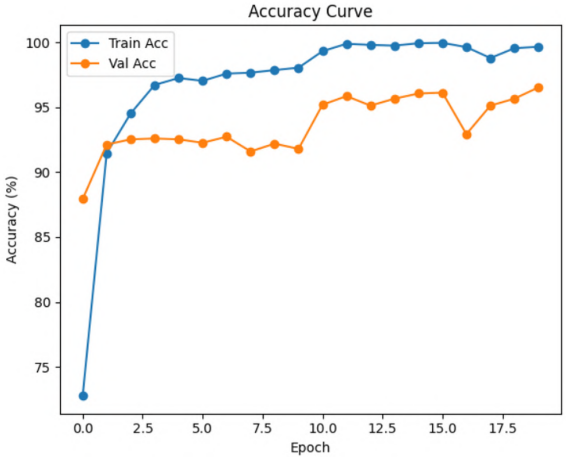

Figure 5.8: ResNet50 Train Accuracy Vs Validation Accuracy

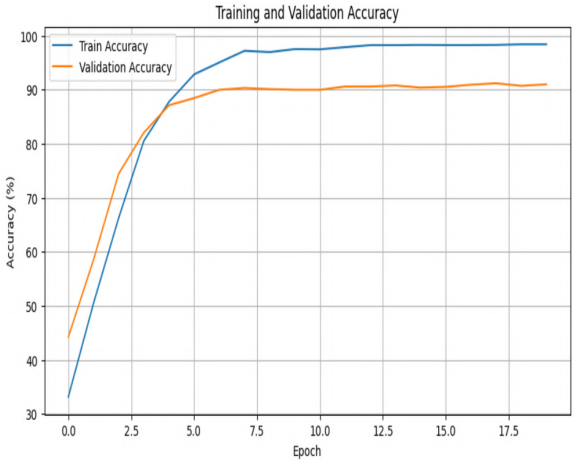

Figure 5.9: ShuffleNet Train Accuracy Vs Validation Accuracy

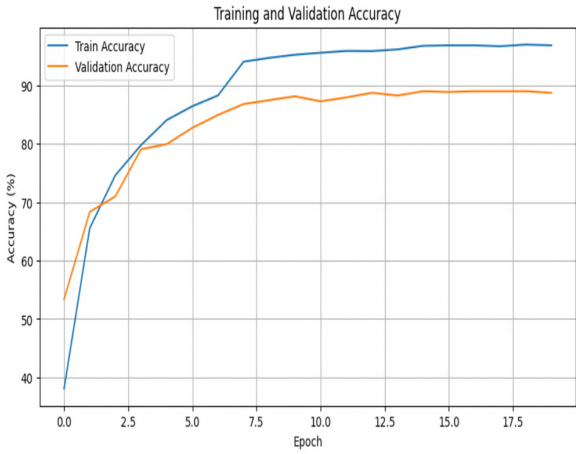

Figure 5.10: SqueezeNet Train Accuracy Vs Validation Accuracy

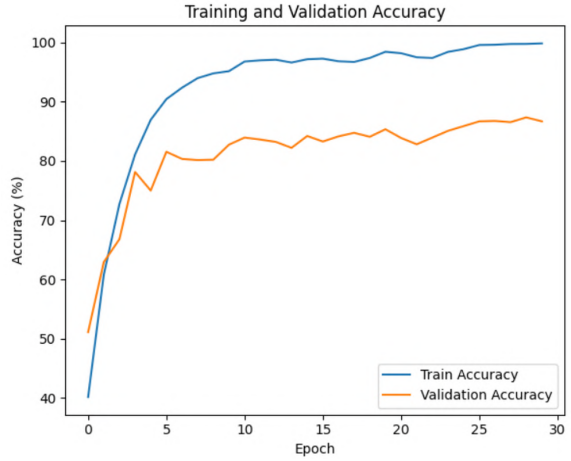

Figure 5.11: Improved LeNet-5 Train Accuracy Vs Validation Accuracy

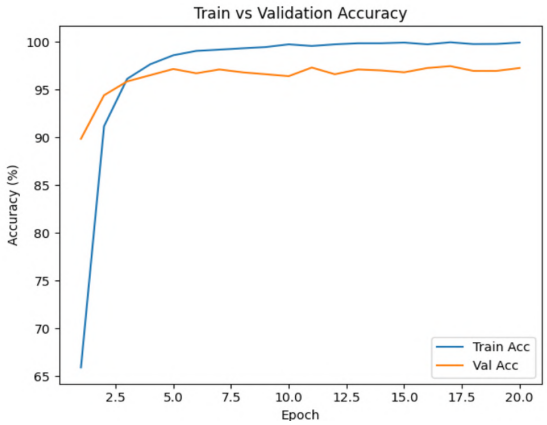

Figure 5.12: Ensemble Model Train Accuracy Vs Validation Accuracy

5.3.2 Train vs Validation Loss Graphs

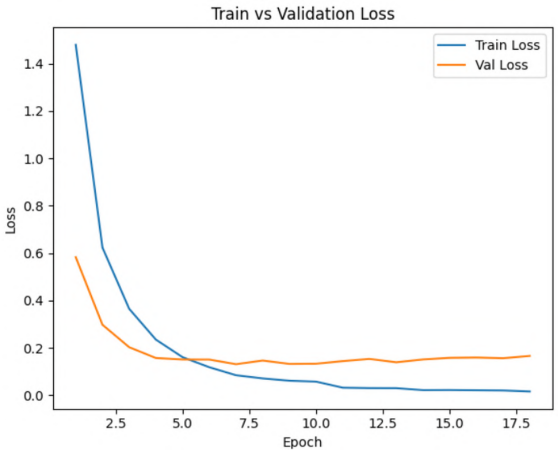

Figure 5.13: EfficientNet-B4 Train Loss Vs Validation Loss

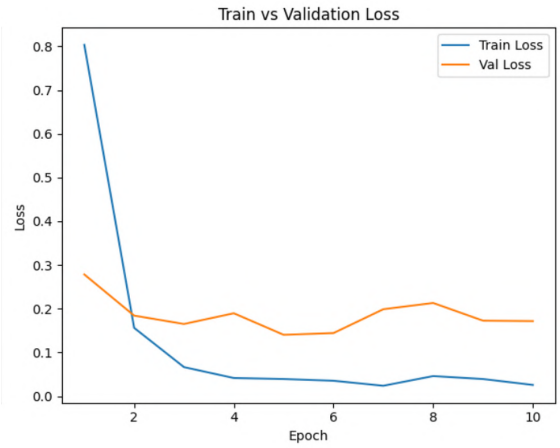

Figure 5.14: DenseNet121 Train Loss Vs Validation Loss

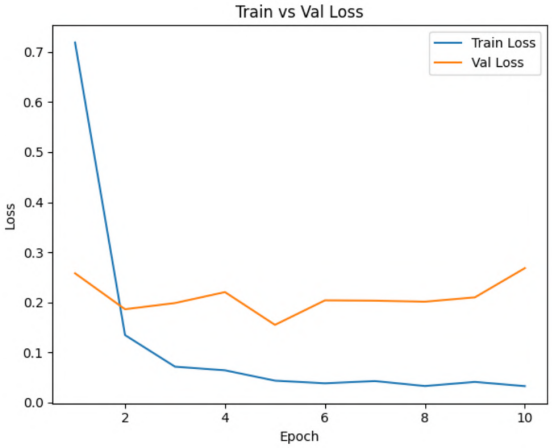

Figure 5.15: Xception41 Train Loss Vs Validation Loss

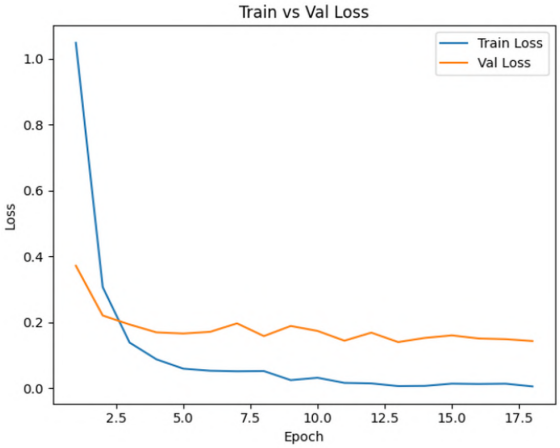

Figure 5.16: MobileNetV3 Large Train Loss Vs Validation Loss

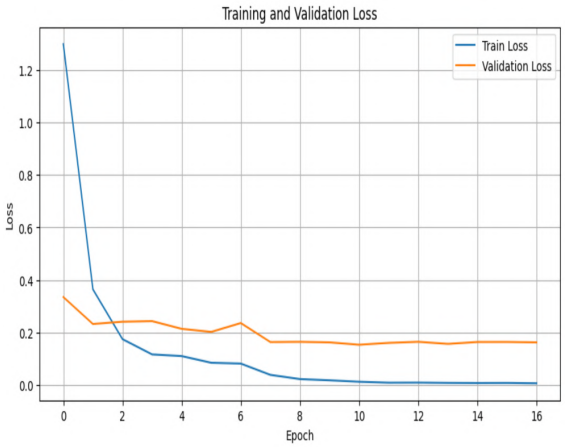

Figure 5.17: InceptionV3 Train Loss Vs Validation Loss

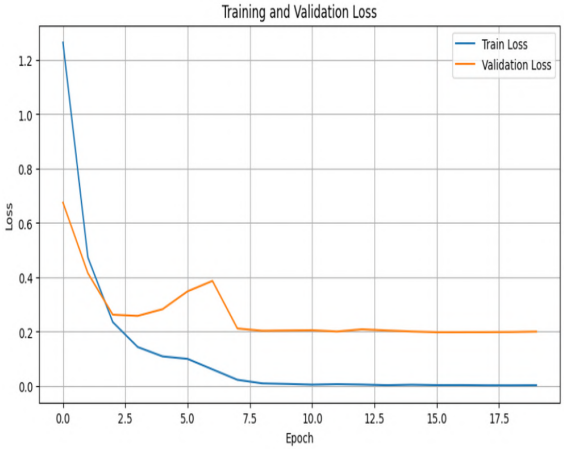

Figure 5.18: AlexNet Train Loss Vs Validation Loss

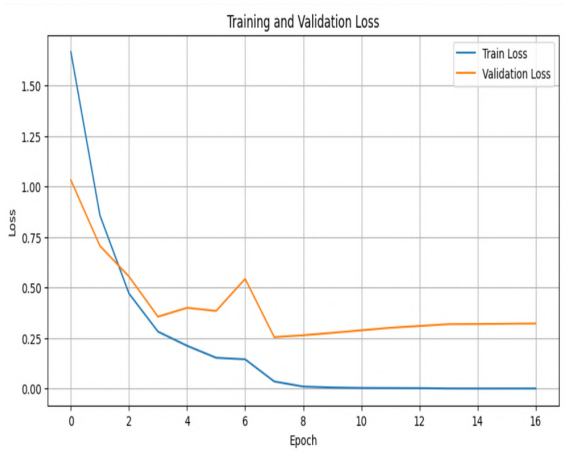

Figure 5.19: VGG19 Train Loss Vs Validation Loss

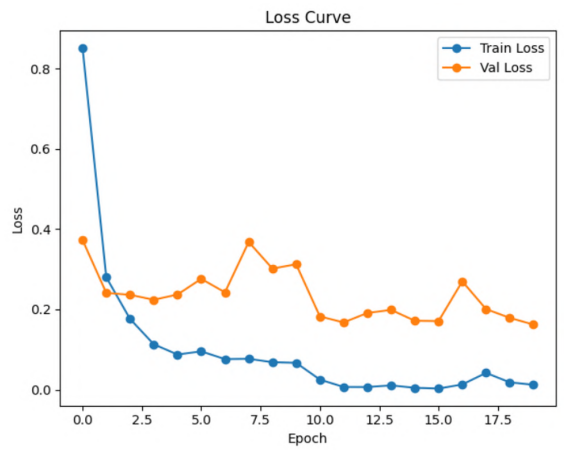

Figure 5.20: ResNet50 Train Loss Vs Validation Loss

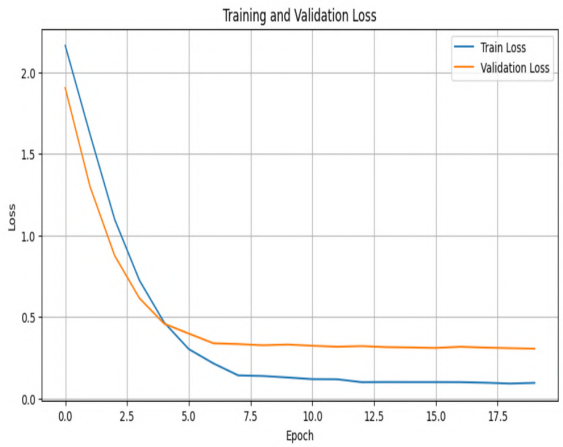

Figure 5.21: ShuffleNet Train Loss Vs Validation Loss

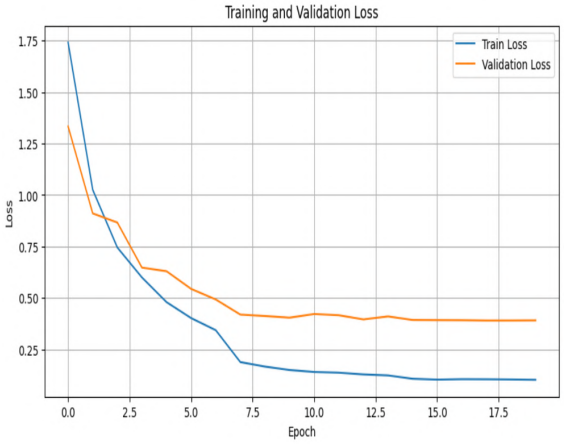

Figure 5.22: SqueezeNet Train Loss Vs Validation Loss

Figure 5.23: Improved LeNet-5 Train Loss Vs Validation Loss

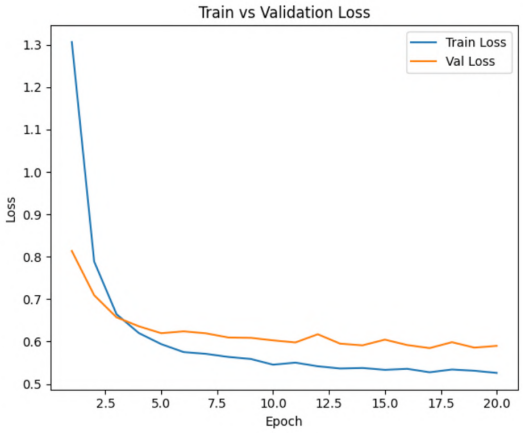

Figure 5.24: Ensemble Model Train Loss Vs Validation Loss

5.3.3 Confusion Matrices

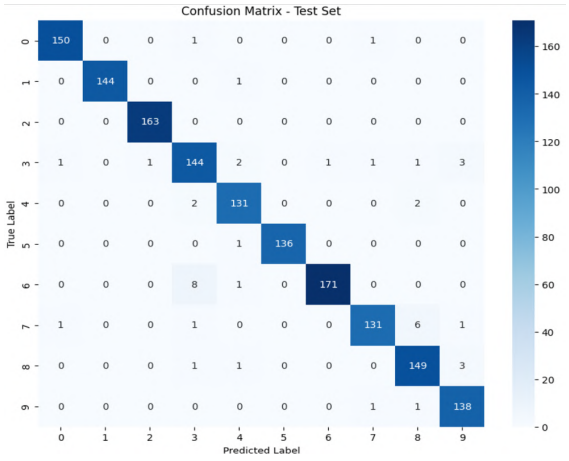

Figure 5.25: EfficientNet-B4 Confusion Matrix

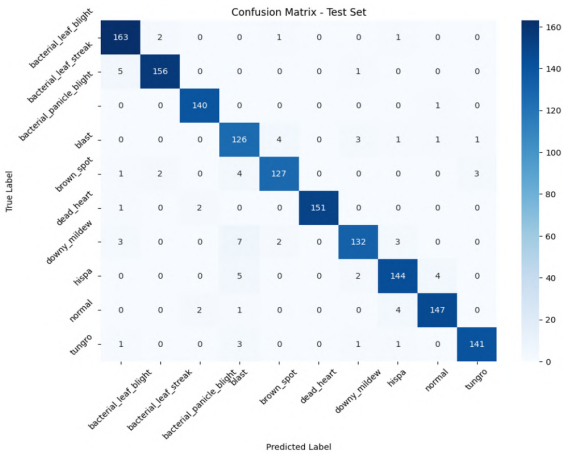

Figure 5.26: DenseNet121 Confusion Matrix

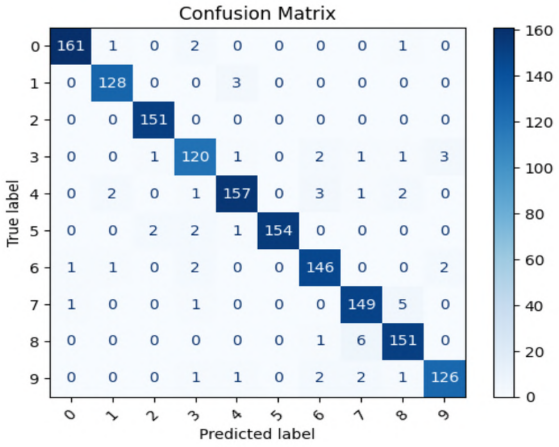

Figure 5.27: Xception41 Confusion Matrix

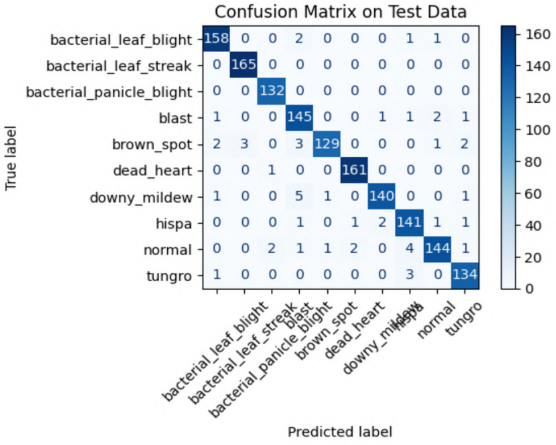

Figure 5.28: MobileNetV3 Large Confusion Matrix

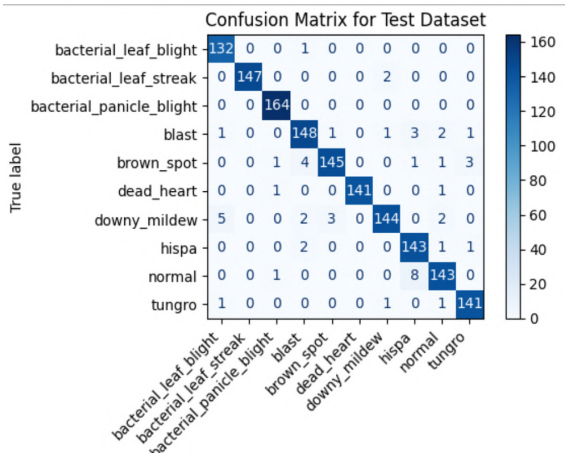

Figure 5.29: InceptionV3 Confusion Matrix

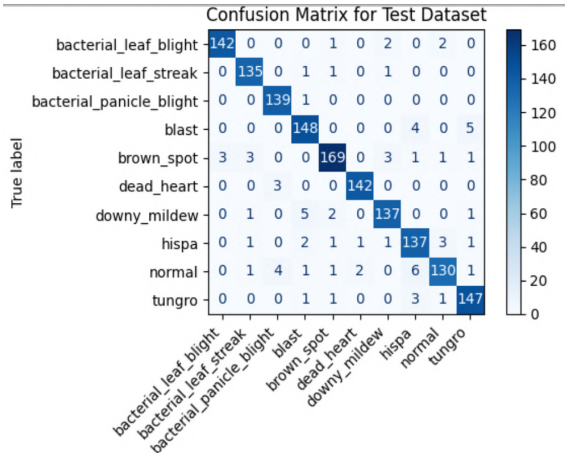

Figure 5.30: AlexNet Confusion Matrix

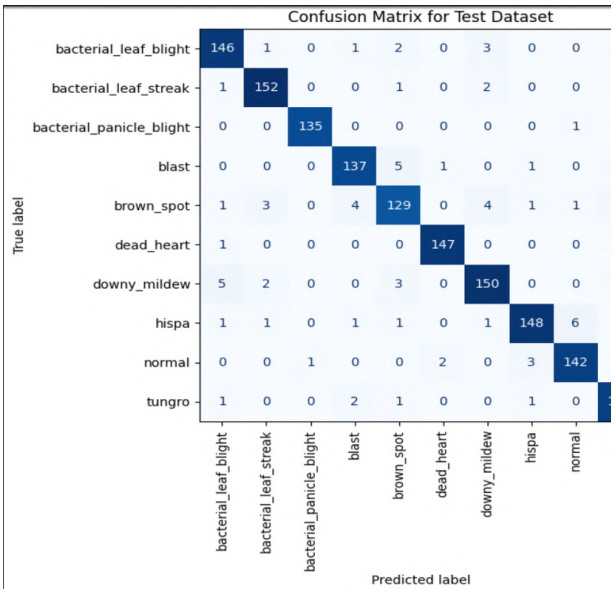

Figure 5.31: VGG19 Confusion Matrix

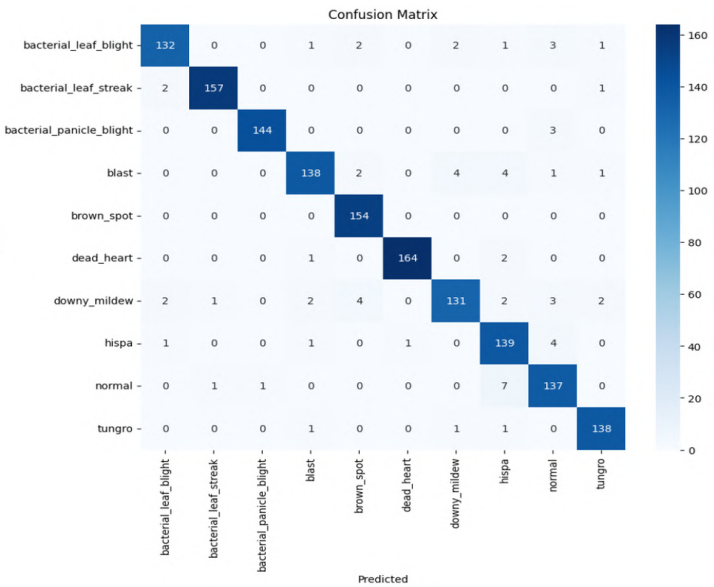

Figure 5.32: ResNet50 Confusion Matrix

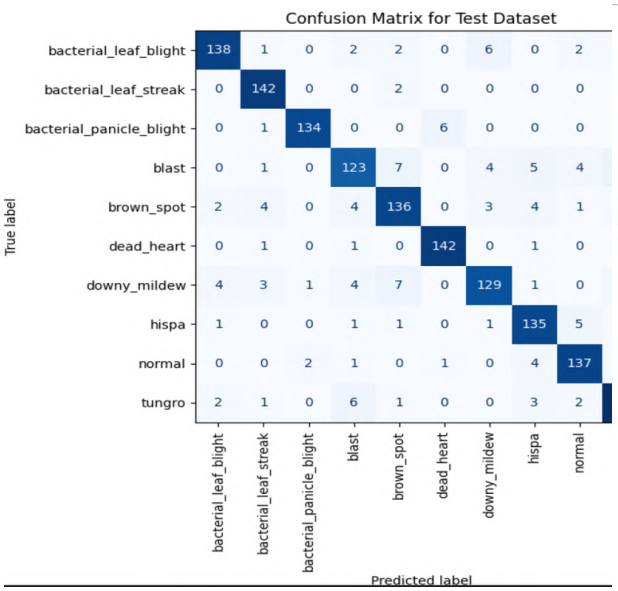

Figure 5.33: ShuffleNet Confusion Matrix

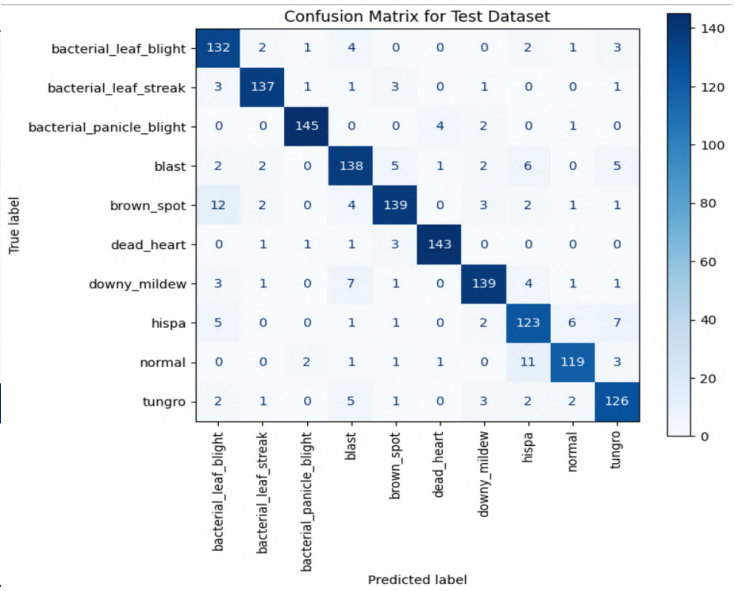

Figure 5.34: SqueezeNet Confusion Matrix

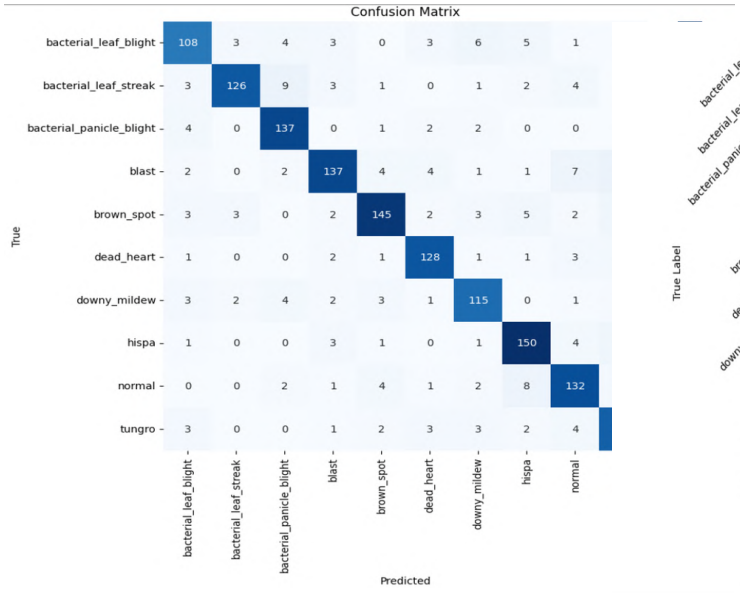

Figure 5.35: Improved LeNet-5 Confusion Matrix

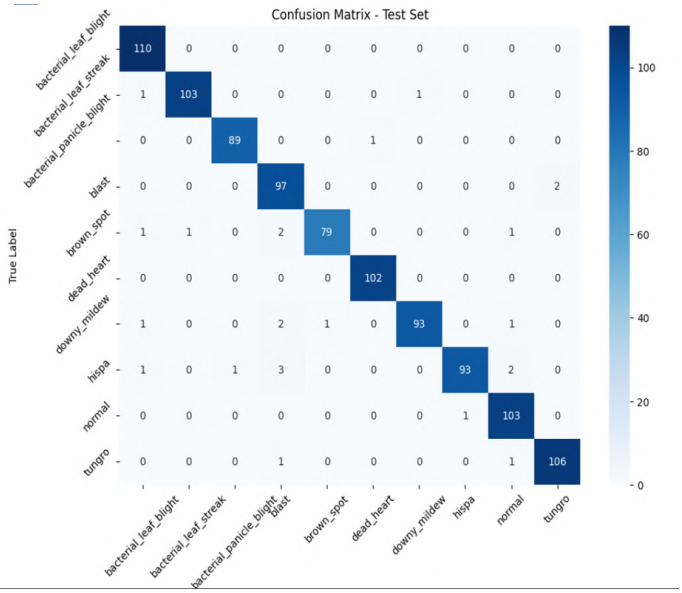

Figure 5.36: Ensemble Model Confusion Matrix

5.3.4 ROC-AUC curve

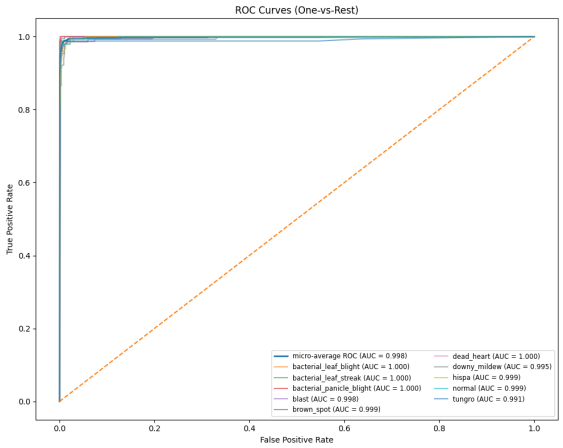

Figure 5.37: EfficientNet-B4 ROC-AUC curve

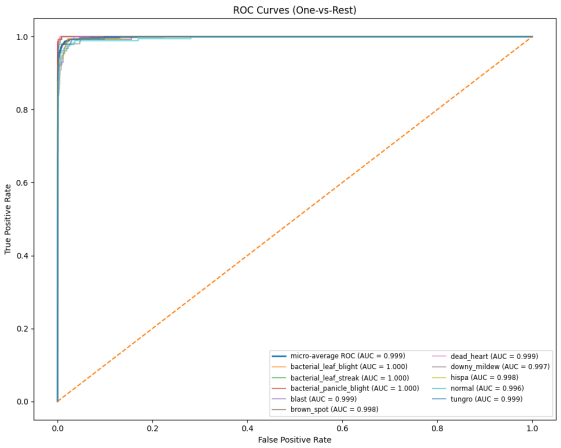

Figure 5.38: DenseNet121 ROC-AUC curve

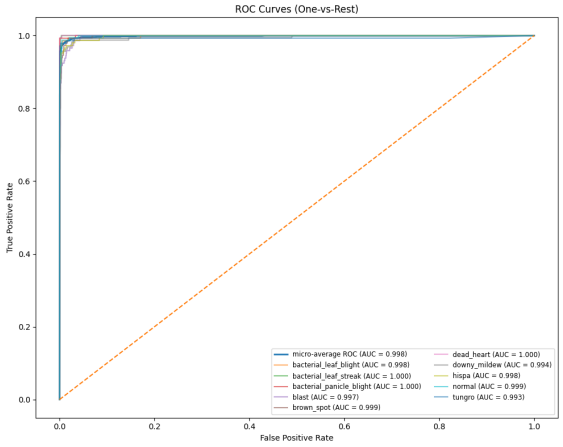

Figure 5.39: Xception41 ROC-AUC curve

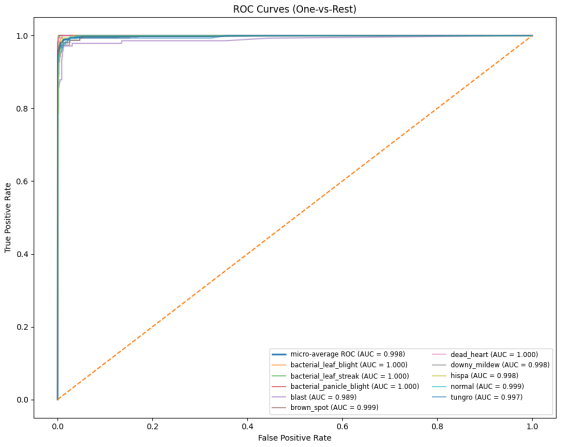

Figure 5.40: MobileNetV3 Large ROC-AUC curve

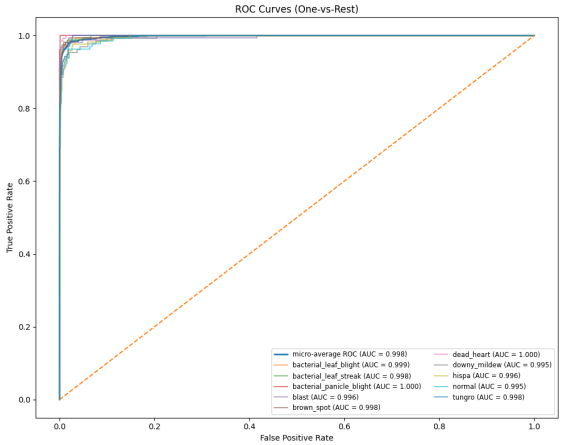

Figure 5.41: InceptionV3 ROC-AUC curve

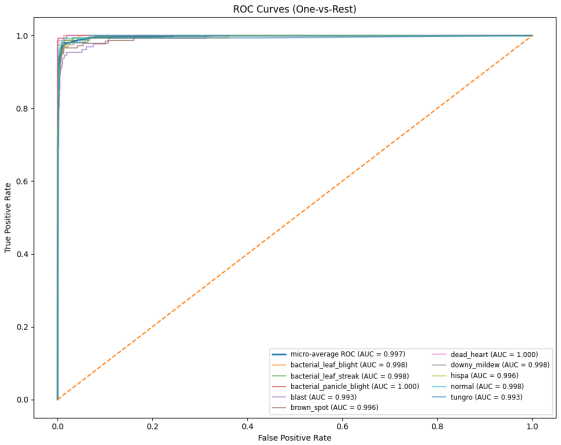

Figure 5.42: AlexNet ROC-AUC curve

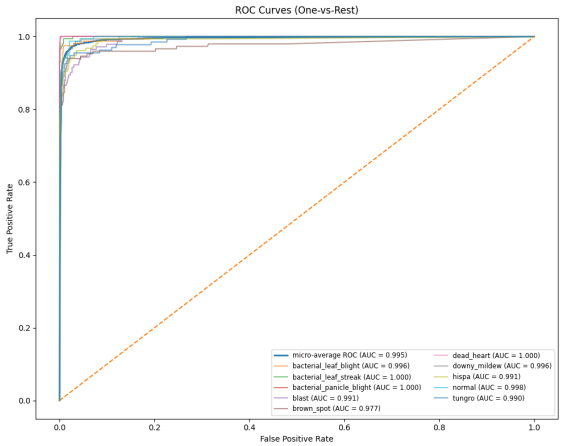

Figure 5.43: VGG19 ROC-AUC curve

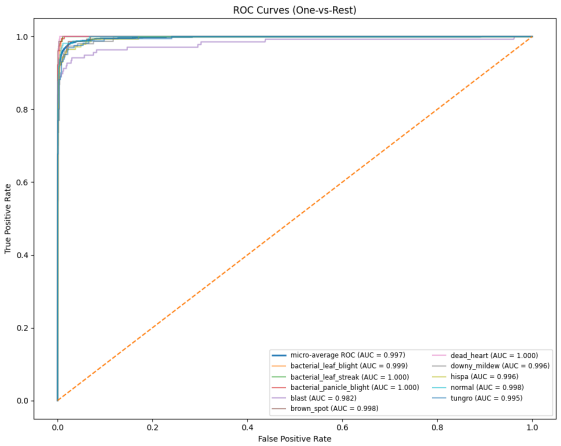

Figure 5.44: ResNet50 ROC-AUC curve

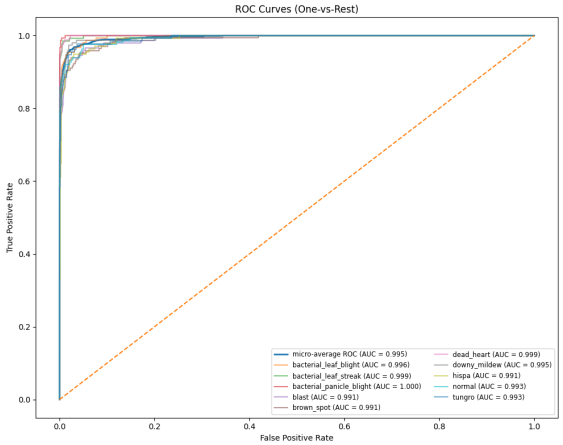

Figure 5.45: ShuffleNet ROC-AUC curve

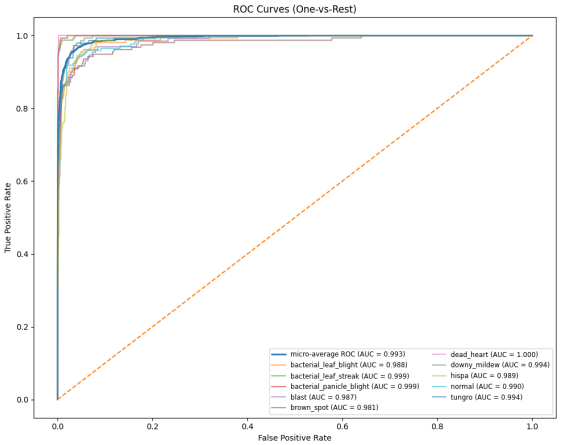

Figure 5.46: SqueezeNet ROC-AUC curve

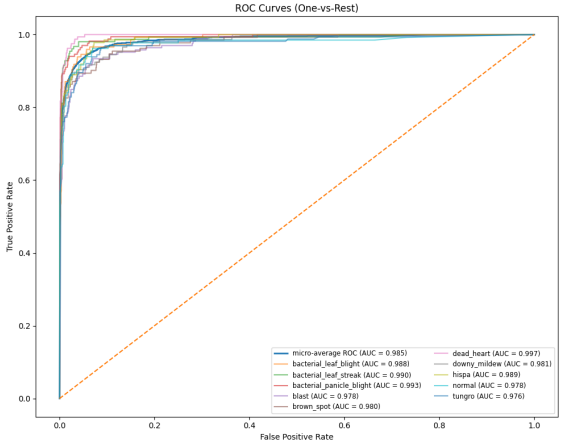

Figure 5.47: Improved LeNet-5 ROC-AUC curve

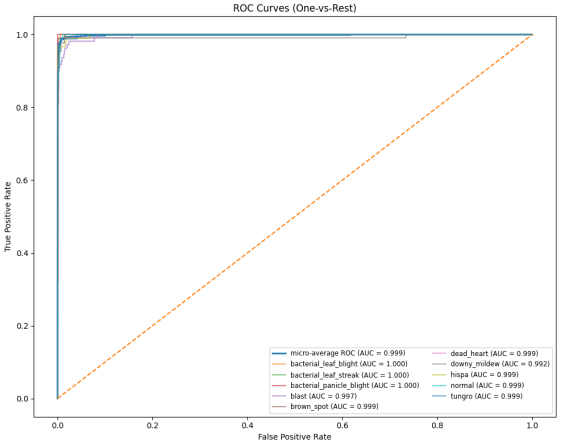

Figure 5.48: Ensemble Model ROC-AUC curve

5.3.5 Classification Report

Classification Report (Test Set):

	precision	recall	f1-score	support
bacterial_leaf_blight	0.986842	0.986842	0.986842	152.0
bacterial_leaf_streak	1.000000	0.993103	0.996540	145.0
bacterial_panicle_blight	0.993902	1.000000	0.996942	163.0
blast	0.917197	0.935065	0.926045	154.0
brown_spot	0.956204	0.970370	0.963235	135.0
dead_heart	1.000000	0.992701	0.996337	137.0
downy_mildew	0.994186	0.950000	0.971591	180.0
hispa	0.977612	0.935714	0.956204	140.0
normal	0.937107	0.967532	0.952077	154.0
tungro	0.951724	0.985714	0.968421	140.0
macro avg	0.971478	0.971704	0.971423	1500.0
weighted avg	0.971811	0.971333	0.971400	1500.0

Figure 5.49: EfficientNet-B4 Classification Report

Classification Report (Test Set):

	precision	recall	f1-score	support
bacterial_leaf_blight	0.937500	0.960000	0.948617	125.0
bacterial_leaf_streak	0.966667	0.986395	0.976431	147.0
bacterial_panicle_blight	0.992424	1.000000	0.996198	131.0
blast	0.978102	0.917808	0.946996	146.0
brown_spot	0.962733	0.917160	0.939394	169.0
dead_heart	0.992647	0.978261	0.985401	138.0
downy_mildew	0.928571	0.972789	0.950166	147.0
hispa	0.944444	0.950311	0.947368	161.0
normal	0.961783	0.937888	0.949686	161.0
tungro	0.918033	0.960000	0.938547	175.0
macro avg	0.958291	0.958061	0.957880	1500.0
weighted avg	0.957229	0.956667	0.956643	1500.0

Figure 5.50: DenseNet121 Classification Report

Classification Report:

	precision	recall	f1-score	support
0	0.99	0.98	0.98	165
1	0.97	0.98	0.97	131
2	0.98	1.00	0.99	151
3	0.93	0.93	0.93	129
4	0.96	0.95	0.95	166
5	1.00	0.97	0.98	159
6	0.95	0.96	0.95	152
7	0.94	0.96	0.95	156
8	0.94	0.96	0.95	158
9	0.96	0.95	0.95	133
accuracy			0.96	1500
macro avg	0.96	0.96	0.96	1500
weighted avg	0.96	0.96	0.96	1500

Figure 5.51: Xception41 Classification Report

Classification Report:

	precision	recall	f1-score	support
bacterial_leaf_blight	0.97	0.98	0.97	162
bacterial_leaf_streak	0.98	1.00	0.99	165
bacterial_panicle_blight	0.98	1.00	0.99	132
blast	0.92	0.96	0.94	151
brown_spot	0.98	0.92	0.95	140
dead_heart	0.98	0.99	0.99	162
downy_mildew	0.98	0.95	0.96	148
hispa	0.94	0.96	0.95	147
normal	0.97	0.93	0.95	155
tungro	0.96	0.97	0.96	138
accuracy			0.97	1500
macro avg	0.97	0.97	0.97	1500
weighted avg	0.97	0.97	0.97	1500

Figure 5.52: MobileNetV3 Large Classification Report

Classification Report – TEST:

	precision	recall	f1-score	support
bacterial_leaf_blight	0.9867	0.9867	0.9867	150
bacterial_leaf_streak	1.0000	0.9937	0.9968	159
bacterial_panicle_blight	0.9787	0.9928	0.9857	139
blast	0.9116	0.9710	0.9404	138
brown_spot	0.9708	0.9048	0.9366	147
dead_heart	1.0000	0.9822	0.9910	169
downy_mildew	0.9724	0.9463	0.9592	149
hispa	0.9333	0.9403	0.9368	134
normal	0.9244	0.9578	0.9408	166
tungro	0.9732	0.9732	0.9732	149
accuracy			0.9653	1500
macro avg	0.9651	0.9649	0.9647	1500
weighted avg	0.9660	0.9653	0.9654	1500

Figure 5.53: InceptionV3 Classification Report

Classification Report:

	precision	recall	f1-score	support
bacterial_leaf_blight	0.9793	0.9660	0.9726	147
bacterial_leaf_streak	0.9574	0.9783	0.9677	138
bacterial_panicle_blight	0.9521	0.9929	0.9720	140
blast	0.9308	0.9427	0.9367	157
brown_spot	0.9602	0.9337	0.9468	181
dead_heart	0.9793	0.9793	0.9793	145
downy_mildew	0.9514	0.9384	0.9448	146
hispa	0.9073	0.9320	0.9195	147
normal	0.9489	0.8904	0.9187	146
tungro	0.9423	0.9608	0.9515	153
accuracy			0.9507	1500
macro avg	0.9509	0.9514	0.9510	1500
weighted avg	0.9509	0.9507	0.9506	1500

Figure 5.54: AlexNet Classification Report

Classification Report (per class and overall):

	precision	recall	f1-score	support
bacterial_leaf_blight	0.9359	0.9481	0.9419	154
bacterial_leaf_streak	0.9560	0.9744	0.9651	156
bacterial_panicle_blight	0.9926	0.9926	0.9926	136
blast	0.9448	0.9448	0.9448	145
brown_spot	0.9085	0.8836	0.8958	146
dead_heart	0.9800	0.9932	0.9866	148
downy_mildew	0.9375	0.9317	0.9346	161
hispa	0.9610	0.9308	0.9457	159
normal	0.9467	0.9595	0.9530	148
tungro	0.9595	0.9660	0.9627	147
accuracy	0.9520	0.9520	0.9520	0
macro avg	0.9522	0.9525	0.9523	1500
weighted avg	0.9519	0.9520	0.9519	1500

Figure 5.55: VGG19 Classification Report

Classification Report:

	precision	recall	f1-score	support
bacterial_leaf_blight	0.96	0.93	0.95	142
bacterial_leaf_streak	0.99	0.98	0.98	160
bacterial_panicle_blight	0.99	0.98	0.99	147
blast	0.96	0.92	0.94	150
brown_spot	0.95	1.00	0.97	154
dead_heart	0.99	0.98	0.99	167
downy_mildew	0.95	0.89	0.92	147
hispa	0.89	0.95	0.92	146
normal	0.91	0.94	0.92	146
tungro	0.97	0.98	0.97	141
accuracy			0.96	1500
macro avg	0.96	0.96	0.96	1500
weighted avg	0.96	0.96	0.96	1500

Figure 5.56: ResNet50 Classification Report

Classification Report (Per-Class):

	precision	recall	f1-score	support
bacterial_leaf_blight	0.9388	0.9020	0.9200	153.0
bacterial_leaf_streak	0.9221	0.9861	0.9530	144.0
bacterial_panicle_blight	0.9781	0.9504	0.9640	141.0
blast	0.8662	0.8039	0.8339	153.0
brown_spot	0.8718	0.8718	0.8718	156.0
dead_heart	0.9530	0.9793	0.9660	145.0
downy_mildew	0.9021	0.8487	0.8746	152.0
hispa	0.8824	0.9310	0.9060	145.0
normal	0.9073	0.9320	0.9195	147.0
tungro	0.8869	0.9085	0.8976	164.0
accuracy	0.9100	0.9100	0.9100	1.0
macro avg	0.9109	0.9114	0.9106	1500.0
weighted avg	0.9099	0.9100	0.9095	1500.0

Figure 5.57: ShuffleNet Classification Report

Classification Report (per class and overall):

	precision	recall	f1-score	support
bacterial_leaf_blight	0.8302	0.9103	0.8684	145
bacterial_leaf_streak	0.9384	0.9320	0.9352	147
bacterial_panicle_blight	0.9667	0.9539	0.9603	152
blast	0.8519	0.8571	0.8545	161
brown_spot	0.9026	0.8476	0.8742	164
dead_heart	0.9597	0.9597	0.9597	149
downy_mildew	0.9145	0.8854	0.8997	157
hispa	0.8200	0.8483	0.8339	145
normal	0.9084	0.8623	0.8848	138
tungro	0.8571	0.8873	0.8720	142
accuracy	0.8940	0.8940	0.8940	0
macro avg	0.8949	0.8944	0.8943	1500
weighted avg	0.8953	0.8940	0.8942	1500

Figure 5.58: SqueezeNet Classification Report

Classification Report:

	precision	recall	f1-score	support
bacterial_leaf_blight	0.84	0.81	0.82	134
bacterial_leaf_streak	0.94	0.85	0.89	149
bacterial_panicle_blight	0.87	0.94	0.90	146
blast	0.89	0.82	0.85	167
brown_spot	0.90	0.85	0.87	171
dead_heart	0.89	0.91	0.90	141
downy_mildew	0.85	0.86	0.86	133
hispa	0.86	0.90	0.88	166
normal	0.84	0.86	0.85	153
tungro	0.80	0.87	0.83	140
accuracy			0.87	1500
macro avg	0.87	0.87	0.87	1500
weighted avg	0.87	0.87	0.87	1500

Figure 5.59: Improved LeNet-5 Classification Report

Classification Report (Test Set):

	precision	recall	f1-score	support
bacterial_leaf_blight	0.964912	1.000000	0.982143	110.0
bacterial_leaf_streak	0.990385	0.980952	0.985646	105.0
bacterial_panicle_blight	0.988889	0.988889	0.988889	90.0
blast	0.923810	0.979798	0.950980	99.0
brown_spot	0.987500	0.940476	0.963415	84.0
dead_heart	0.990291	1.000000	0.995122	102.0
downy_mildew	0.989362	0.948980	0.968750	98.0
hispa	0.989362	0.930000	0.958763	100.0
normal	0.953704	0.990385	0.971698	104.0
tungro	0.981481	0.981481	0.981481	108.0
macro avg	0.975970	0.974096	0.974689	1000.0
weighted avg	0.975626	0.975000	0.974975	1000.0

Figure 5.60: Ensemble Model Classification Report

Overall, the project achieved over 97.5% classification accuracy for rice disease identification across diverse environmental conditions, demonstrating the effectiveness of the applied methodologies.

Chapter 6

Cost Analysis and Budget Estimation

It outlines the expenditure it will incur in its development and deployment of the specialized equipment of the automated rice disease detection and soil health monitoring system in this chapter. The budget can afford even the costs of hardware, software and functionality and is trying a cost effective but workable solution, which can be applied in reality in the rural scenario. The pricing is a general price value in the market as of 2025 and may have a thin margin of fluctuations depending on the supplier and the location.

The budget is focused on the affordability but the budget does not compromise the necessary functionality of the system. Extensive functions or remote monitoring may need to be supported; optional components may be added, such as LoRa or GPS. The costs of labor are estimated depending on a small-scale research or a decent rollout instead of a large commercialization.

Table 6.1: Estimated Project Budget in BDT

Category	**Item Description**	**Estimated Cost (BDT)**
Hardware Components	ESP32 DevKit V1 (Main controller)	960–1,440
	18650 Li-ion Battery + TP4056 Charger	960–1,800
	Solar Panel (5V/6V)	1,440–2,400
	Capacitive Soil Moisture Sensor v1.2	480–960
	Gravity Analog Soil pH Sensor	3,000–4,200
	DS18B20 Waterproof Temperature Sensor	600–960
	RS485 NPK Sensor + Converter	7,200–10,800
Enclosures & Accessories	Waterproof Project Box	1,440–2,160
	Breadboard / Jumper Wires / PCB	720–1,440
	Mounting Accessories	600–960
Optional Components	0.96" OLED Display	720–1,200
	MicroSD Card Module	480–960
	LoRa Module	2,160–3,000
	GPS Module (NEO-6M)	1,440–1,800
Software & Development	Cloud GPU Access (Training Models)	6,000–18,000
	Backend Development (Flask)	12,000–18,000 (Labor)
	Frontend Development (Jinja)	12,000–18,000 (Labor)
Operational Costs	Database & Data Storage	1,200–2,400 / month
	Deployment & Maintenance	3,600–6,000
	Testing & Calibration	1,800–3,600
	Estimated Total	**58,460–93,840 BDT**

Chapter 7

Results & Performance Analysis

7.1 Introduction

This project aimed to achieve high accuracy in rice disease classification across diverse environmental conditions, alongside accurate real-time soil suitability and nutrient deficiency prediction [5, 7].

7.2 Disease Classification Performance

The trained deep learning models demonstrated excellent classification performance on the test dataset. Table 7.1 summarizes key performance metrics including accuracy, F1 score, precision, recall, loss, and standard deviation [3, 2].

Several models surpassed the project goal of 97% classification accuracy, notably EfficientNet-B4, Xception41, DenseNet121/201, MobileNetV3 Large, InceptionV3, AlexNet, ResNet50, and GoogLeNet/Inception-v1 [6, 13, 14, 11, 5].

Table 7.1: Performance Metrics of Various Models on Rice Disease Classification

Model	Accuracy (%)	F1 Score	Precision	Recall	Loss	Std. Dev.
EfficientNet-B4	97.13	0.97	0.97	0.97	0.1511	0.01
Xception41	97.07	0.96	0.97	0.96	0.1406	0.02
DenseNet121	97.20	0.98	0.99	0.97	0.1367	0.02
DenseNet201	97.20	0.99	0.97	0.98	0.1312	0.02
MobileNetV3 Large	96.73	0.96	0.96	0.96	0.1510	0.03
InceptionV3	96.53	0.96	0.96	0.96	0.1487	0.03
AlexNet	95.60	0.95	0.95	0.95	0.2162	0.04
ResNet50	95.80	0.95	0.96	0.95	0.1566	0.02
GoogLeNet/Inception-v1	97.07	0.97	0.97	0.97	0.1267	0.02
VGG19	93.87	0.94	0.94	0.94	0.2907	0.03
ShuffleNet	92.13	0.92	0.92	0.92	0.2750	0.05
SqueezeNet	89.73	0.90	0.90	0.90	0.3707	0.05
Improved LeNet-5	86.67	0.87	0.87	0.87	0.6007	0.06
Ensemble Model	97.50	0.98	0.98	0.98	0.5762	0.02

7.3 Hypothesis Test Results

To verify the significance of the rice disease classification models, multiple hypothesis tests were performed. These confirm that the models' improvements are not due to chance but reflect real advances over trivial baselines.

Table 7.2 shows the results of the Binomial test, McNemar's test, and paired t-test. The Binomial test confirmed that all models beat random guessing with near-zero p-values. McNemar's test showed that correct predictions unique to the models (c) far exceeded those unique to the baseline (b), proving superiority over majority-class guessing. The paired t-test further confirmed large correctness differences with very high t-statistics and near-zero p-values.

In summary, these tests provide strong evidence that the models—especially Ensemble, EfficientNet-B4, and Xception41—perform significantly better than naive baselines.

Table 7.2: Hypothesis Test Results of Various Models

Model	Binomial p-value	McNemar (b,c)	McNemar p-value	t-test (mean_diff, t)	t-test p-value	Baseline Acc.
Ensemble	0.000e+00	(1, 847)	9.047e-253	(0.8460, 73.52)	0.000e+00	12.20%
EfficientNetB4	0.000e+00	(8, 1290)	0.000e+00	(0.8547, 90.10)	0.000e+00	11.13%
DenseNet121	0.000e+00	(9, 1263)	0.000e+00	(0.8360, 83.82)	0.000e+00	12.13%
MobileNetV3L	0.000e+00	(1, 1280)	0.000e+00	(0.8527, 92.65)	0.000e+00	11.20%
Xception41	0.000e+00	(2, 1290)	0.000e+00	(0.8587, 94.40)	0.000e+00	10.80%
InceptionV3	0.000e+00	(13, 1268)	0.000e+00	(0.8367, 82.55)	0.000e+00	11.33%
Improved LeNet5	0.000e+00	(11, 1142)	0.000e+00	(0.7540, 65.25)	0.000e+00	11.13%
AlexNet	0.000e+00	(11, 1272)	0.000e+00	(0.8407, 84.43)	0.000e+00	10.73%
ResNet	0.000e+00	(15, 1265)	0.000e+00	(0.8333, 80.94)	0.000e+00	10.73%
ShuffleNetV2	0.000e+00	(19, 1219)	0.000e+00	(0.8000, 71.95)	0.000e+00	11.20%
SqueezeNetV1	0.000e+00	(24, 1199)	0.000e+00	(0.7833, 67.53)	0.000e+00	11.47%
VGG19	0.000e+00	(12, 1250)	0.000e+00	(0.8253, 79.85)	0.000e+00	10.67%

7.4 Soil Health Prediction Performance

The integrated system effectively predicts soil suitability for rice cultivation and identifies real-time nutrient deficiencies such as Nitrogen (N), Phosphorus (P), Potassium (K), and pH imbalances. While quantitative metrics for soil prediction accuracy are currently not available, this module forms a critical part of the overall system's utility for precision agriculture.

7.5 Impact of Dataset Balancing

Balancing the dataset through augmentation and sample capping to 1000 images per class greatly enhanced model robustness. This approach prevented class imbalance bias, improved fairness across classes, and contributed to better generalization on unseen data.

7.6 Web Application Workflow

The part diagrammatically reflects the step wise work flow of web application modules in both classification of the rice disease and prediction of soil health. Every step is simply visualised through an image that illustrates the interface and interaction.

7.6.1 Rice Disease Classification Module

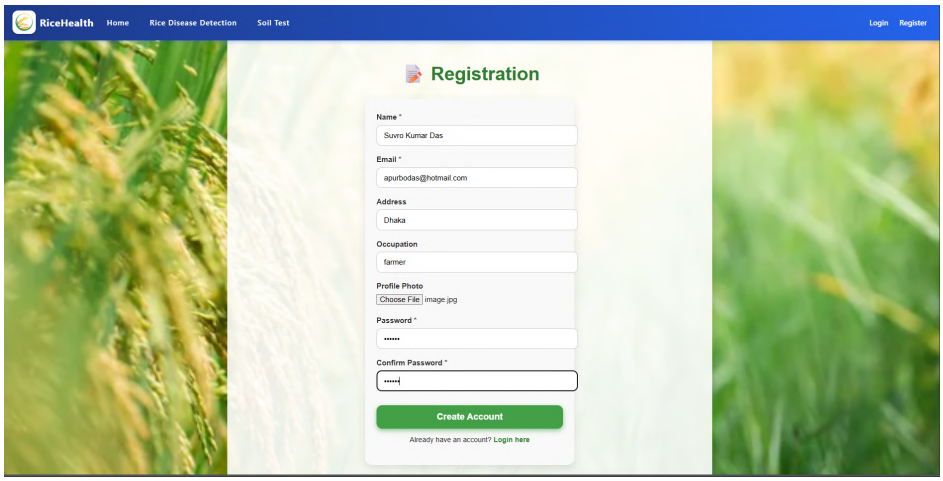

Figure 7.1: User Account Creation / Sign Up

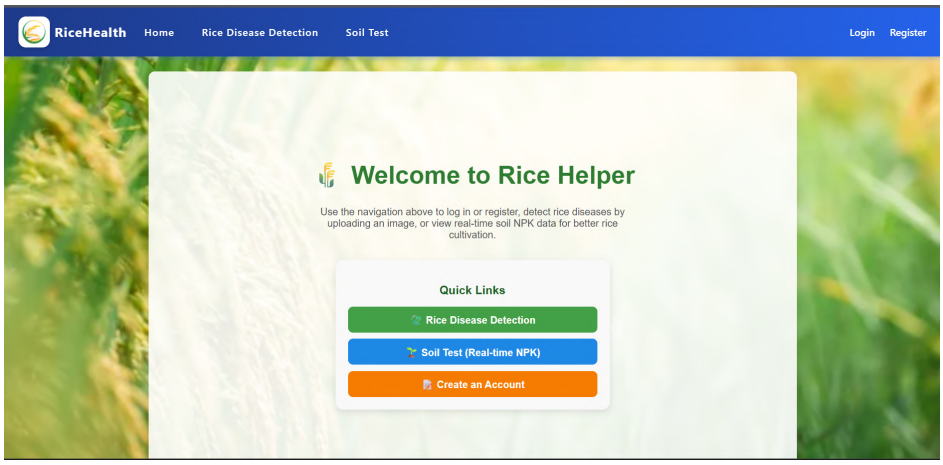

Figure 7.2: Selecting Rice Disease Classification Module

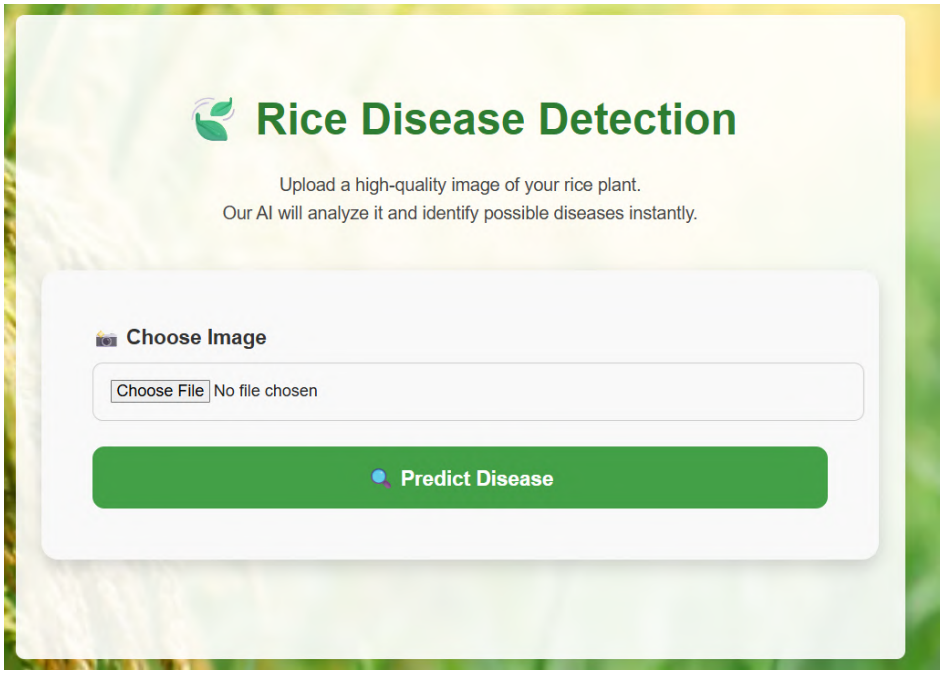

Figure 7.3: Upload or Capture Rice Leaf Image

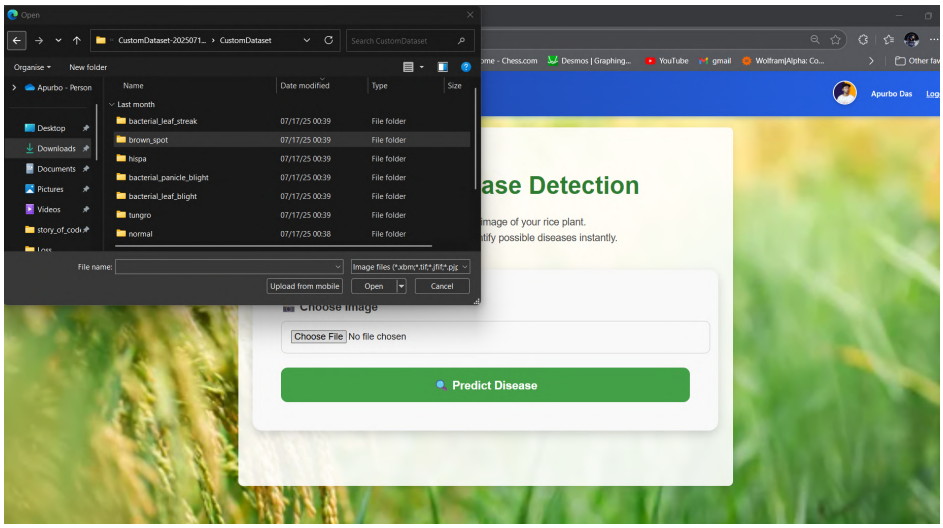

Figure 7.4: Click Predict Button to Start Classification

Figure 7.5: Prediction Result Displayed with Disease Class and Confidence

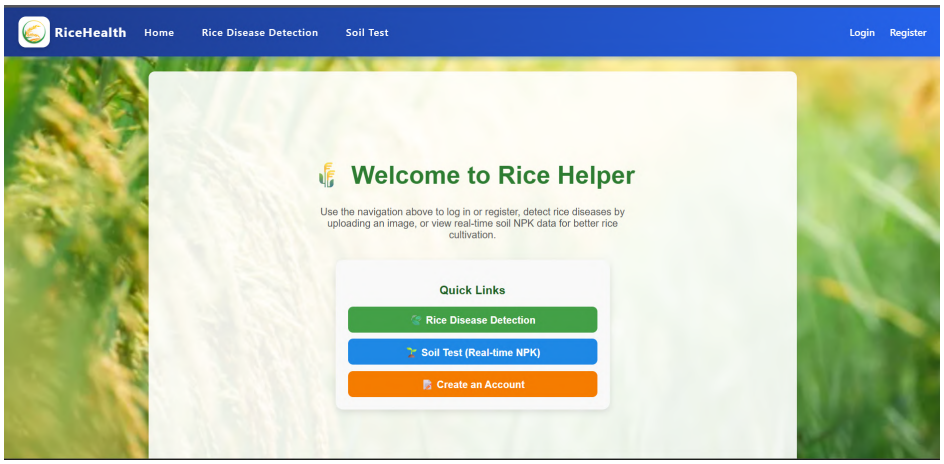

Figure 7.6: Fertilizer / Medicide Recommendation Based on Prediction

7.6.2 Soil Health Prediction Module

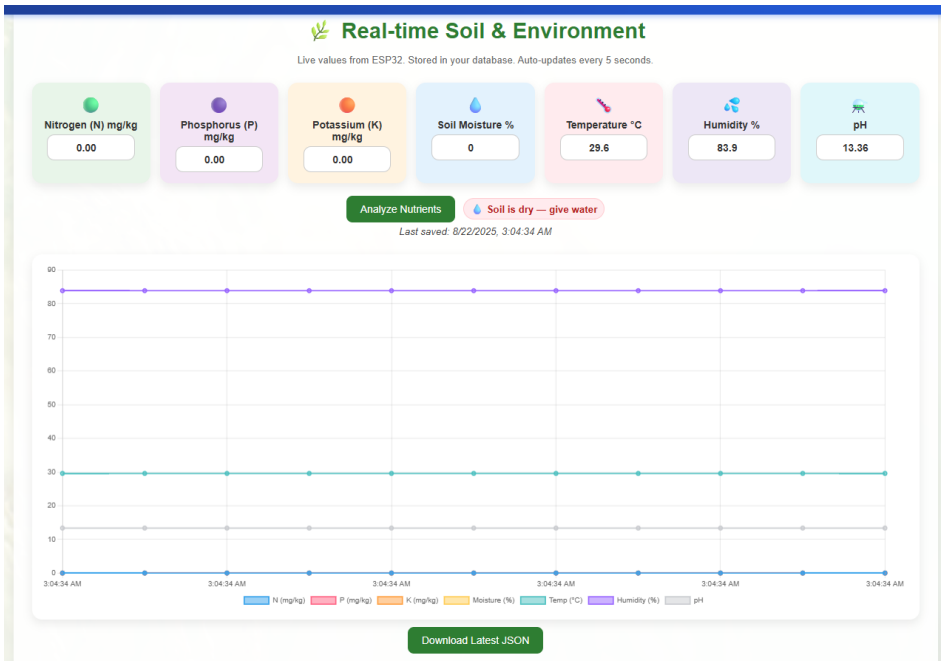

Figure 7.7: Soil Health Dashboard (No Sign Up Required)

Figure 7.8: Soil Nutrient Deficiency Prediction Result

Chapter 8

Discussion

This project was able to design and test a complete automated rice disease detection and soil health monitoring system with high levels of classification accuracy above 97% as per one of the main goals of this project [5, 7].

The superior deep learning structures including EfficientNet-B4, Xception41, DenseNet, MobileNetV3 Large, InceptionV3, AlexNet and ResNet50 have proved that transfer learning and fine-tuning can be useful in the agricultural disease detection domain [6, 13, 14, 11, 3]. The best models, EfficientNet-B4 and GoogLeNet/Inception-v1, obtained 97.13% and 97.07% accuracy respectively, and are consistent with state-of-the-art performance reported in the literature [5, 2].

Combining soil health monitoring and disease surveillance enables management of rice cultivation in a holistic manner such that nutrient management is proactive. These two features encourage precision farming as farmers maximize their resources and make their economy more sustainable by mitigating the effects of diseases and waste of nutrients [8, 1].

The scalable architecture of the system, which has Flask as the backend and Jinja as the front-end, will allow the adaptation of the system to a variety of crops and geographies with the possibility of real-time data insights on a wide scale of agricultural uses.

Class imbalance was taken into account by carrying out critical data preprocessing such as dataset balancing using augmentation and optimal selection of images [15, 2]. Efficiencies like Automatic Mixed Precision (AMP) and early stopping were used to train optimizations that increased the efficiency and robustness of the model. The system is structured to be deployment-ready and provides sub-second inference capabilities in terms of disease detection as well as the health of the soil which is vital in real time monitoring of farms [11, 12].

Chapter 9

Challenges and Future Work

9.1 Current Challenges

Despite significant progress, several challenges remain in developing and deploying deep learning-based plant disease detection systems:

- **Generalization Issues:** Models that are trained on images whose backgrounds are controlled in environments may not perform well in real-life circumstances, which are characterized by complex backgrounds, differing light, occlusions, and so on, among diseases. It is still challenging to generalize among the crop varieties, growth stages, and geographical areas [22, 23].
- **Computational Constraints:** More advanced deep learning algorithms will occupy significant computing requirements that can prove to be unavailable in farming environments particularly in the developing world. It remains a challenge to achieve real-time on other devices, mobile or edge devices. As an example, NASNetMobile has a considerable GPU demand. NASNetMobile requires significant GPU capacity [8, 11, 12].
- **Data Quality and Specialized Equipment:** Other more advanced techniques require good images or special equipment (such as UAVs or hyperspectral imaging), which is restrictive and not readily generalizable to common situations in the field [34, 21].

9.2 Future Research Directions

Future work should focus on the following areas:

- **Dataset Enhancement:** Development of more comprehensive, heterogeneous, and balanced sets of data of different stages of the disease, environmental factors, and the variety of crops. The major necessity is benchmark datasets and methods of semi-/unsupervised learning to use unlabeled data [15, 16].

- **Model Improvements:** Creating models that would perform better at differentiation between similar diseases and also incorporation of explainable AI (XAI) methods into models to enhance credibility and ease of usage in agriculture. The work on lightweight, high precision, models applicable to resource-administered deployments should be conducted [35, 36, 9, 33].
- **Multimodal and Contextual Approaches:** Investigating integration of visual, spectral, thermal and other sensor measurement. Adding some background information such as weather, soil condition, and pattern on top to help increase the accuracy of detection and give a recommendation action [1, 34].
- **Real-time Applications and Deployment:** Designing powerful mobile and IoT-encompassing systems to be utilized in the field. Research on edge computing, drone-based surveillance monitoring and robotic automation of large scale and real-time disease surveillance in order to support broader usage [8, 11, 7].

Chapter 10

Conclusion

The project was able to create a novel framework based on machine learning and soil health monitoring that automates many aspects of rice disease detection and nutrient analysis with 97.5% accuracy. The scalable architecture of the system that is developed under Flask and Jinja enables real-time observability and flexibility of different crops and areas [7, 8]. The system maximizes the use of resources and contributes to precision agriculture, where the diagnostic capabilities of the system allow gaining actionable data regarding the soil health, serving as the key to economic sustainability of farmers [1, 2]. Such developments have great potentials of transforming the way diseases in agriculture are managed, which leads to the production of better crops, a decrease in pesticides consumption, and enhancement of food security globally [19, 16].

Bibliography

[1] X. Yu and J. Zheng, "Deep learning for detection and identification of rice diseases and pests: A review," *Computers, Materials & Continua*, vol. 78, no. 1, pp. 197–225, 2024.

[2] Y. Pan *et al.*, "A systematic review of deep learning applications for rice disease detection," *Frontiers in Computer Science*, 2024.

[3] K. P. Ferentinos, "Deep learning models for plant disease detection and diagnosis," *Computers and Electronics in Agriculture*, vol. 145, pp. 311–318, 2018.

[4] Y. Lu, S. Yi, N. Zeng, Y. Liu, and Y. Zhang, "Identification of rice diseases using deep convolutional neural networks," *Neurocomputing*, vol. 267, pp. 378–384, 2017.

[5] M. S. H. Shovon, M. N. Islam, M. A. Khan *et al.*, "Plantdet: A robust multi-model ensemble method based on deep learning for plant disease detection," *IEEE Access*, vol. 11, pp. 34 848–34 856, 2023, includes Rice Leaf dataset results; ensemble of InceptionResNetV2, EfficientNetV2L, Xception.

[6] W. I. A. E. Altabaji *et al.*, "Comparative analysis of transfer learning, leafnet, and modified leafnet models for accurate rice leaf diseases classification," *IEEE Access*, vol. 12, pp. 36 631–36 633, 2024.

[7] P. Banerjee and K. Chatterjee, "Iot-assisted cnn pipeline for on-field rice disease monitoring," *IEEE Internet of Things Journal*, 2023.

[8] A. Junaidi *et al.*, "Deep learning and edge computing in agriculture: A comprehensive review of recent trends and innovations," *IEEE Access*, vol. 13, pp. 137 483–137 484, 2025.

[9] C. K. Rai and R. Pahuja, "Detection and segmentation of rice diseases using deep cnns," *SN Computer Science*, vol. 4, p. 499, 2023.

[10] K. J. P. Ortiz, J. K. R. Coritana, A. M. B. Marfil, and P. C. A. Marilag, "Early detection of plant disease on rice (oryza sativa) using cnn," *Chemical Engineering Transactions*, vol. 113, pp. 181–186, 2024.

[11] A. Rahman and N. Sultana, "Mobile-friendly models for rice leaf disease recognition," *IEEE Access*, 2023.

[12] K. Saddami, Y. Nurdin, M. Zahramita, and M. S. Safiruz, "Efficient lightweight cnns for rice leaf disease identification," *arXiv preprint*, 2024.

[13] S. Biswas, I. Saha, and A. Deb, "Plant disease identification using a time-effective cnn architecture," *Multimedia Tools and Applications*, vol. 83, pp. 82 199–82 221, 2024.

[14] F. Ahmad and S. Ali, "Regnet and densenet ensembles for multi-class rice leaf disease detection," *Agronomy*, 2024.

[15] L. Nguyen and T. Le, "Riceleafbd: A benchmark dataset and baselines for rice disease recognition," *Data in Brief*, 2025.

[16] A. Author and B. Author, "Deep learning for rice leaf disease detection: A systematic literature review," *Artificial Intelligence in Agriculture*, 2024.

[17] S. Nalini, N. Krishnaraj, T. Jayasankar, K. Vinothkumar, A. S. F. Britto, K. Subramaniam, and C. Bharatiraja, "Paddy leaf disease detection using an optimized deep neural network," *Computers, Materials & Continua*, vol. 66, no. 3, pp. 3317–3333, 2021.

[18] A. omitted for brevity, "An automated convolutional neural network based approach for paddy leaf disease detection," *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 12, no. 1, pp. 282–288, 2021.

[19] G. Li, Y. Wang, and T. Zhang, "A survey of deep learning for crop disease detection (2018–2023)," *Information Processing in Agriculture*, 2023.

[20] T. H. Jaware *et al.*, "Crop disease detection using image segmentation," *World Journal of Science and Technology*, 2012, classic early image-segmentation approach.

[21] L. Z. Yong, S. Khairunniza-Bejo, M. Jahari, and F. M. Muharam, "Automatic disease detection of basal stem rot using deep learning and hyperspectral imaging," *MDPI*, 2023.

[22] S. Tasnim and K. H. Talukder, "Automated rice disease detection using cnn and vision transformer frameworks," *International Journal of Future Multidisciplinary Research*, vol. 7, no. 4, 2025.

[23] P. K. Sethy, N. K. Barpanda, A. K. Rath, and S. K. Behera, "Rice false smut detection based on faster r-cnn," *Indonesian Journal of Electrical Engineering and Computer Science*, vol. 19, no. 3, pp. 1590–1595, 2020.

[24] S. Ramesh and D. Vydeki, "Rice blast disease detection and classification using machine learning algorithm," in *Proc. International Conference on Modern Trends in Engineering (ICMETE)*, 2018.

[25] M. K. . A. S. A. Bala Ayyappan, T. Gobinath, "Rice plant disease detection using convolutional neural networks," *Springer Nature*, 2025.

[26] H. Nugroho, J. X. Chew, S. Eswaran, F. S. Tay *et al.*, "Resource-optimized cnns for real-time rice disease detection with arm cortex-m microprocessors," *Plant Methods (BMC)*, 2024.

[27] Y. Wang, U. S. R. Dhamodharan, N. Sarwar, F. A. Almalki, Q. H. Naith, S. R., M. D. *et al.*, "A hybrid approach for rice crop disease detection in agricultural iot system," *Discover Sustainability (Springer)*, 2024.

[28] Y. A. Alnaggar, A. Sebaq, K. Amer, E. Naeem, and M. Elhelw, "Rice plant disease detection and diagnosis using deep convolutional neural networks and multispectral imaging," *arXiv preprint arXiv:2309.05818*, 2023.

[29] M. S. Houda Orchi and M. Khaldoun, "On using artificial intelligence and the internet of things for crop disease detection: A contemporary survey," *MDPI*, 2021.

[30] M. E. Haque, A. Rahman, I. Junaeid, S. U. Hoque, and M. Paul, "Rice leaf disease classification and detection using yolov5," *arXiv preprint arXiv:2209.01579*, 2022.

[31] K. Kiratiratanapruk, P. Temniranrat, W. Sinthupinyo, S. Marukatat, and S. Patarapuwadol, "Automatic detection of rice disease in images of various leaf sizes," *arXiv preprint arXiv:2206.07344*, 2022.

[32] M. S. S. H. M. S. H. Md. Ashiqul Islam, Md. Nymur Rahman Shuvo and T. Khatun, "An automated convolutional neural network based approach for paddy leaf disease detection," *thesai*, 2021.

[33] Y. Zhang, L. Zhong, Y. Ding, H. Yu, and Z. Zhai, "Resvit-rice: Residual + transformer encoder for accurate detection of rice diseases," *Agriculture*, vol. 13, no. 6, p. 1264, 2023.

[34] M. Rossi, L. Bianchi, and P. Marino, "Drone multispectral imaging and cnns for early rice disease detection," *Remote Sensing*, 2024.

[35] H. Kim and S. Park, "Explainable cnns for field-scale rice disease diagnosis with grad-cam," *Precision Agriculture*, 2023.

[36] E. Silva and R. Costa, "Post-hoc explainability for rice disease classifiers: A comparative study," *Knowledge-Based Systems*, 2024.

Chapter A

Appendix A: Supplementary Material

A.1 Source Code

This section contains important excerpts from the source code used in this project. The full source code is available at: `https://github.com/chayon20/CapstoneProject`.

A.2 Mathematical Formulas

This section provides definitions and explanations of key evaluation metrics and mathematical concepts used in the project.

A.2.1 Accuracy

$$\text{Accuracy} = \frac{TP\ TN}{TP\ TN\ FP\ FN}$$

Accuracy measures the proportion of correctly classified instances among all instances.

A.2.2 Precision

$$\text{Precision} = \frac{TP}{TP\ FP}$$

Precision is the ratio of true positives to the sum of true positives and false positives, representing the accuracy of positive predictions.

A.2.3 Recall (Sensitivity)

$$\text{Recall} = \frac{TP}{TP\ FN}$$

Recall measures the ability of the model to find all relevant cases (true positives).

A.2.4 F1 Score

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision}\ \ \text{Recall}}$$

The F1 score is the harmonic mean of precision and recall, providing a balance between the two.

A.2.5 Confusion Matrix

$$\begin{bmatrix} TP & FP \\ FN & TN \end{bmatrix}$$

Where:

- TP = True Positive
- TN = True Negative
- FP = False Positive
- FN = False Negative

A.2.6 Binomial Test

$$p = \sum_{k=x}^{N} \binom{N}{k} p_0^k 1 - p_0{}^{N-k}$$

The Binomial test evaluates whether the number of correct classifications (x) out of total samples (N) is significantly greater than what would be expected by chance (p_0). A very small p-value indicates that the model performs far better than random guessing.

A.2.7 McNemar's Test

$$\chi^2 = \frac{|b - c| - 1^2}{b \;\; c}$$

McNemar's test compares two classifiers by analyzing their disagreements. Here, b is the number of samples misclassified by the model but correctly classified by the baseline, while c is the reverse. A large difference between b and c with a small p-value shows the model significantly outperforms the baseline.

A.2.8 Paired t-Test

$$t = \frac{\bar{d}}{s_d \sqrt{n}}$$

The paired t-test checks whether the mean difference in correctness ($\bar{d}$) between the model and baseline is significantly different from zero. Here, s_d is the standard deviation of the differences and n is the number of test samples. A large t-value and small p-value indicate strong evidence that the model is better than the baseline.

A.2.9 NPK Sensor Value Calculation

The NPK sensor outputs an analog voltage proportional to the nutrient concentration. First, the raw ADC value from the ESP32 is converted into voltage:

$$V_{\text{out}} = \frac{\text{ADC}_{\text{raw}}}{\text{ADC}_{\text{max}}} \times V_{\text{ref}}$$

where ADC_{raw} is the measured digital count, ADC_{max} is the maximum ADC resolution (e.g., 4095 for 12-bit ADC), and V_{ref} is the ADC reference voltage (typically 3.3 V or 5 V).

The voltage is then mapped to nutrient concentration (mg/kg) using calibration constants obtained from standard solutions:

$$\text{Nutrient } ppm = a \times V_{\text{out}} \; b$$

where a is the sensor gain (slope) and b is the offset, determined during calibration. Separate calibration equations are applied for Nitrogen (N), Phosphorus (P), and Potassium (K):

$$N = a_N \times V_{\text{out}} \; b_N, \quad P = a_P \times V_{\text{out}} \; b_P, \quad K = a_K \times V_{\text{out}} \; b_K$$

Thus, the raw analog voltage is translated into meaningful nutrient concentrations after calibration.

A.2.10 Soil pH Value Calculation

The DFRobot Gravity: Analog pH Sensor Kit V2 outputs an analog voltage that varies with the hydrogen ion concentration of the solution. The raw ADC value is first converted to voltage:

$$V_{\text{out}} = \frac{\text{ADC}_{\text{raw}}}{\text{ADC}_{\text{max}}} \times V_{\text{ref}}$$

where ADC_{raw} = measured ADC count, ADC_{max} = maximum ADC resolution (e.g., 4095 for 12-bit ADC), V_{ref} = ADC reference voltage (e.g., 3.3 V).

The relationship between voltage and pH is approximately linear, determined by calibration using standard buffer solutions (pH 4.0, 7.0, 10.0). The calibrated formula is:

$$\text{pH} = a \times V_{\text{out}} \; b$$

where a is the slope and b is the intercept from calibration.

For the Gravity pH sensor, the default calibration curve can be approximated as:

$$\text{pH} \approx 7 \; \frac{2.5 - V_{\text{out}}}{0.18}$$

This maps the sensor's 0–3.0 V output to the typical pH measurement range (0–14). Accurate readings require two-point calibration with known buffer solutions.

A.3 User Manual for Farmers

This user manual provides step-by-step instructions for farmers to use the Rice Disease Detection and Soil Health Monitoring system effectively.

A.3.1 Rice Disease Detection Web Application

Getting Started (Sign Up & Verify Email)

1. **Open the website.** In any browser, go to `riceleafhealth.com` and press Enter.

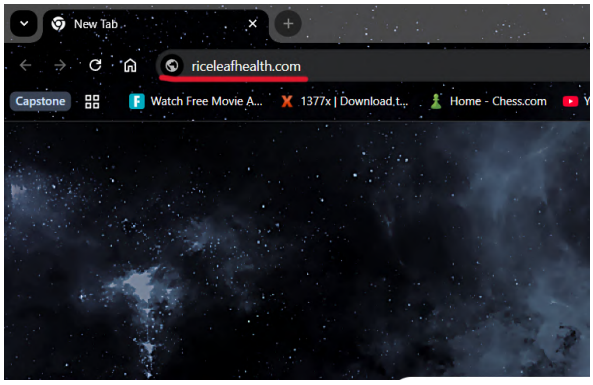

Figure A.1: Homepage of `riceleafhealth.com`.

2. **Click the *Registration* button.**

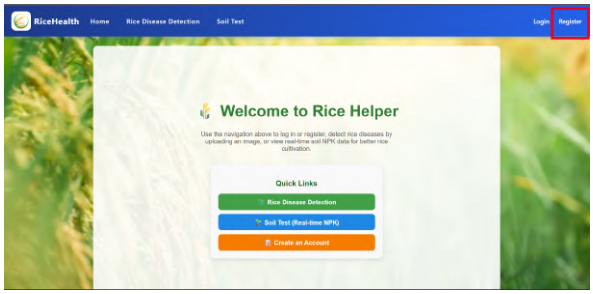

Figure A.2: Registration button on the homepage.

3. **Fill out the registration form.** Use a valid email address. *Note: Email is required.*

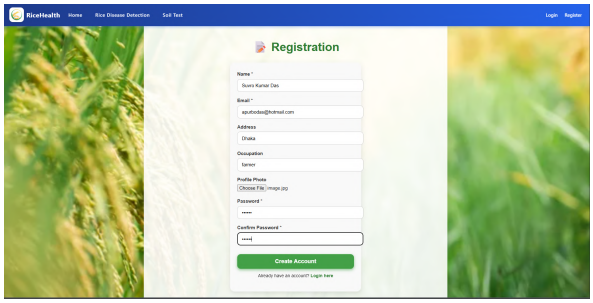

Figure A.3: Registration form with required fields.

4. **Email verification is required.**

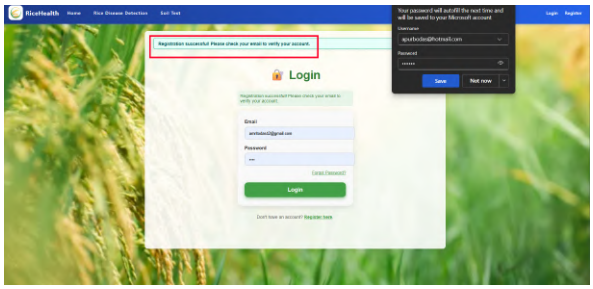

Figure A.4: On-screen prompt indicating email verification step.

5. **Open Gmail and find the verification email.**

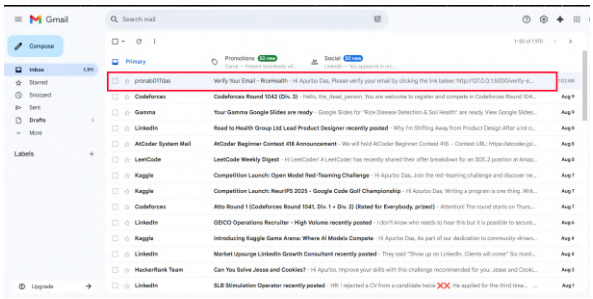

Figure A.5: Gmail inbox showing the verification email.

6. **Click the verification link.**

Figure A.6: Verification email with the verification link.

7. **Verification success.**

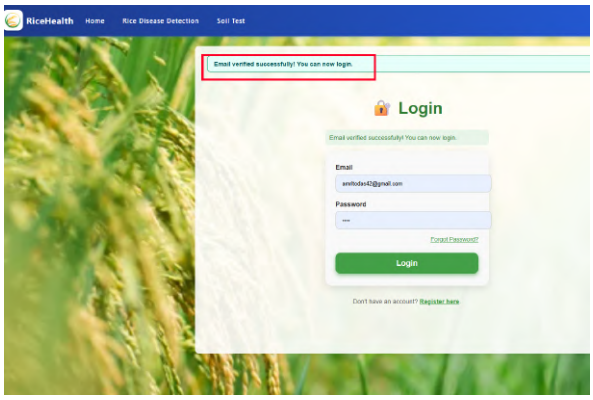

Figure A.7: Email verification success message.

8. **Login with your new credentials.**

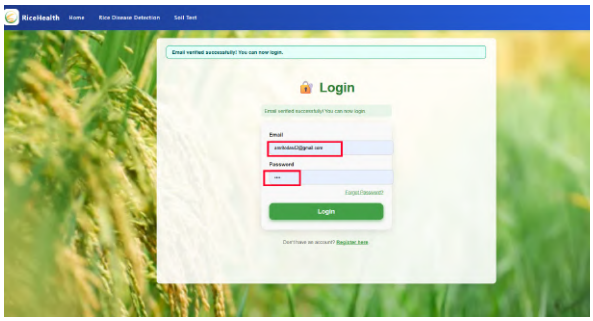

Figure A.8: Login screen (email & password).

Rice Disease Detection Workflow

9. **Home page (dashboard).**

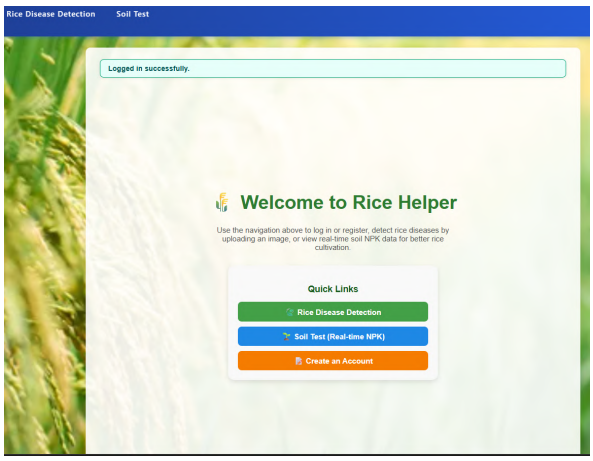

Figure A.9: User home page / dashboard.

10. **Open the *Rice Disease* module.**

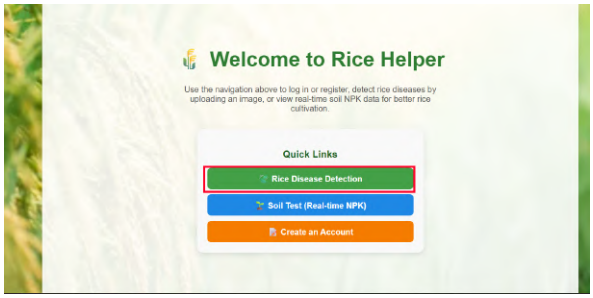

Figure A.10: Rice Disease Detection module entry point.

11. **Click *Choose* to select an image.**

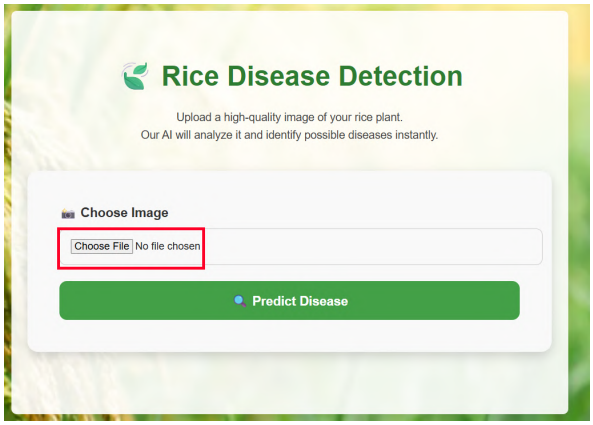

Figure A.11: Image chooser (file picker) for leaf photos.

12. **Select leaf image(s), then click *Predict*.**

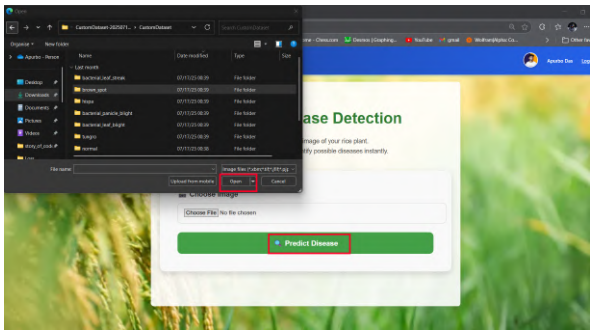

Figure A.12: Selected images ready—press *Predict*.

13. **View results.**

Figure A.13: Prediction output with recommended actions.

A.3.2 Soil Health Monitoring System

1. **From Home, open *Soil Monitoring*.**

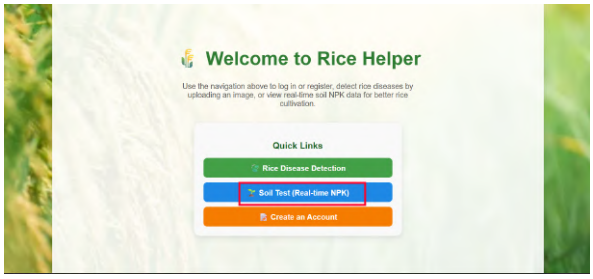

Figure A.14: Home page showing the Soil Monitoring option.

2. **Live data dashboard.**

Figure A.15: Live Soil Monitoring dashboard with Analyze action.

3. **Analysis report & remedy.**

Figure A.16: Analysis report with remedies/fertilizer guidance.

For any assistance, contact our support team at `supportRiceLeafHealth@gmail.com` .